

Платформа подбора роботизированных решений

Сопроводительная документация

29 сентября 2026 г.

Содержание

О платформе

Начало работы

- Обзор возможностей
- Развёртывание
- Проверка демонстрационного сценария

Как это устроено

- Архитектура
- Подбор роботов
- Экономическая модель
- Как читать результат
- Симуляция
- Данные и каталог

Руководства

- Руководство пользователя
- Руководство администратора
- Роли и доступ
- Сопровождение

Справочник

- Переменные окружения
- Программный интерфейс
- Глоссарий
- Библиотеки и источники данных
- Ограничения и план

Для разработчиков

- Разработка
- ADR 0001. Снимки, варианты, задания и события
- ADR 0002. Симуляция, карта и очередь
- ADR 0003. Операции и синхронизация
- ADR 0004. Экономика по денежному потоку

О платформе

Платформа оценивает, окупится ли внедрение роботов на складе, в аэропорту или в медицинском учреждении. Она подбирает подходящие модели из каталога, считает экономику покупки и аренды и сверяет расчёт с имитацией работы роботов на плане склада.

Результат платформы предварительный. Он помогает быстро проверить гипотезу и сравнить варианты. Обследование объекта и полное технико-экономическое обоснование он не заменяет.

С чего начать

Задача	Раздел
Понять, что умеет платформа	Обзор возможностей
Поднять платформу на своей машине	Развёртывание
Проверить основные функции за 10 шагов	Проверка демонстрационного сценария
Посчитать проект	Руководство пользователя
Вести каталог и нормативы	Руководство администратора
Выложить платформу в сеть и хранить данные	Сопровождение
Изменить код	Разработка

Из чего состоит документация

- **Начало работы:** обзор, развёртывание, проверка демо.
- **Как это устроено:** архитектура, подбор, экономическая модель, симуляция, данные каталога.
- **Руководства:** пользователь, администратор, роли, сопровождение.
- **Справочник:** переменные окружения, программный интерфейс, глоссарий, библиотеки и источники данных, ограничения.
- **Для разработчиков:** правила изменения кода и принятые архитектурные решения.

Форматы документации

- Сайт: команда `mdbook build` в каталоге `docs/`.
- Один файл: PDF и DOCX. Как их собрать, написано в разделе [Разработка](#).
- Исходные файлы Markdown в каталоге `docs/`.

Команды запуска и список переменных окружения лежат в [README репозитория](#) и в разделах [Развёртывание](#) и [Переменные окружения](#).

Начало работы

Обзор возможностей

Платформа считает экономику внедрения роботов на объектах трёх типов: склад, аэропорт и медицинское учреждение. Для склада она дополнительно имитирует работу роботов на плане и проверяет, что заявленная производительность достижима.

Основные понятия

Полный список терминов приведён в [гlossарии](#).

- **Проект.** Данные об одном объекте: параметры, процессы, карта, варианты и результаты расчётов.
- **Вариант.** Состав флота роботов и способ оплаты. В проекте до шести вариантов, по умолчанию создаются три.
- **Сценарий.** Способ сравнения. Платформа считает три сценария: «База» (текущий процесс без роботов), «Покупка» (оборудование приобретается) и «RaaS» (оборудование арендуется).
- **Запуск.** Неизменяемая запись об одном расчёте или одной симуляции. Запуск хранит входные данные, версии моделей и контрольные суммы, поэтому результат можно воспроизвести.
- **Демо.** Готовый проект, который открывается без входа. Для каждого типа объекта есть свой демо-проект.

Сценарии использования

1. Гость открывает демо, меняет параметры и смотрит страницу «Итог». Проекты гость не создаёт, введённые данные остаются в его браузере.
2. Пользователь создаёт проект, заполняет параметры объекта и процессы, выбирает роботов, получает расчёт и выгружает отчёт в форматах PDF и XLSX.
3. Администратор ведёт каталог роботов, задаёт нормативы расчёта, приглашает пользователей и просматривает журнал действий.

Путь пользователя

Шаг	Вкладка	Результат
1. Выбор объекта	Стартовая страница	Создан проект или открыто демо склада, аэропорта или больницы
2. Параметры	Объект	Заполнены параметры объекта, процессы и карта склада

Шаг	Вкладка	Результат
3. Подбор	Роботы	Получен список моделей с вердиктом и объяснением, роботы выбраны в варианты
4. Расчёт	Расчёт / Итог	Рассчитаны CAPEX, OPEX, эффект, окупаемость и ROI для трёх сценариев
5. Допущения	Расчёт / Допущения	Изменены ключевые допущения, результат пересчитан
6. Симуляция	Расчёт / Симуляция	Проверена производительность флота на карте склада
7. Экспорт	Расчёт / Экспорт	Скачаны отчёты в форматах PDF и XLSX

Степень готовности

Функция	Готовность	Комментарий
Подбор по техническим характеристикам	Полностью	Жёсткие проверки, оценка с весами, объяснение для каждой модели
Экономическая модель	Полностью	Три сценария, денежный поток по годам, чувствительность, нормативы
Отчёты PDF и XLSX	Полностью	Отчёт строится из снимка запуска
Симуляция на карте склада	Полностью	Событийная модель, подбор числа роботов прогонами
Проверка для аэропорта и больницы	Частично	Быстрая проверка без карты, событийной симуляции нет
Каталог роботов	Частично	Технические характеристики заполнены не у всех моделей
Интеграции с WMS, ERP и 1C	Не сделано	Описаны как план развития
3D и ранжирование методами машинного обучения	Не сделано	Не входят в поставку

Ограничения и план развития описаны в разделе [Ограничения и план](#).

Развёртывание

Платформа состоит из трёх контейнеров: базы данных Postgres, сервера API и веб-сервера со статическим приложением. Контейнеры описаны в файле `docker-compose.yml` в корне репозитория.

Требования

Компонент	Требование
Docker и Docker Compose	Docker Engine или Docker Desktop с плагином Compose версии 2
Порты на хосте	80 (интерфейс) и 8080 (API); значения меняются переменными <code>WEB_PORT</code> и <code>API_HOST_PORT</code>
Место на диске	около 1 ГБ для образов и тома базы данных
Доступ в интернет	нужен только при первой сборке образов

Для запуска в контейнерах устанавливать Go, Node.js и Postgres не требуется.

Запуск

1. Клонировать репозиторий и перейти в его корень.
2. Выполните команду:

```
docker compose up --build
```

3. Дождитесь сообщения `listen` в журнале сервера API. При первом запуске API применяет миграции, загружает каталог из файла `data/catalog.csv` и создаёт демонстрационные учётные записи.
4. Откройте в браузере адрес `http://localhost`.

Порты меняются так:

```
WEB_PORT=8088 API_HOST_PORT=18080 docker compose up --build
```

Проверка работы

1. Откройте `http://localhost:8080/health`. Ответ с кодом 200 означает, что процесс API работает.
2. Откройте `http://localhost` и нажмите **Демо Склад**. Должна открыться страница «Объект» с заполненными параметрами.
3. Перейдите на вкладку **Расчёт**. Должна открыться страница «Итог» с таблицей из трёх сценариев.

Если каталог пуст, посев данных не прошёл. Выполните команды из раздела [Сброс данных](#) и запустите платформу снова.

Демонстрационные учётные записи

Учётные записи создаются при старте API, если записей с такими адресами ещё нет.

Роль	Адрес электронной почты	Пароль
Пользователь	<code>user@demo.local</code>	<code>demo-user</code>
Администратор	<code>admin@demo.local</code>	<code>demo-admin</code>

Важно. Смените демонстрационные пароли перед любой публикацией платформы в сети. Порядок описан в разделе [Сопровождение](#).

Гостю вход не нужен: демо-проекты «Демо Склад», «Демо Аэропорт» и «Демо Больница» открываются в браузере без учётной записи.

Сброс данных

Команда ниже удаляет том базы данных. Вместе с ним пропадают все проекты, запуски и учётные записи.

```
docker compose down -v
```

При следующем запуске платформа заново загрузит каталог и создаст демонстрационные учётные записи. Не используйте параметр `-v` на стенде, данные которого нужно сохранить.

Запуск без Docker

Этот способ нужен разработчикам. Потребуется Postgres 18, Go и Node.js.

1. Создайте пустую базу данных и задайте переменные окружения. Пример значений приведён в файле `.env.example`. Для локальной работы установите `COOKIE_SECURE=false` и `PUBLIC_ORIGIN=http://localhost:5173`.

2. Запустите сервер API:

```
cd api && go run ./cmd/api
```

3. Запустите веб-приложение в другом окне терминала:

```
cd web && npm install && npm run dev
```

Vite передаёт запросы `/api` серверу API, поэтому браузер видит один адрес.

4. Соберите браузерное ядро расчёта, иначе проверка карты и симуляция демо-склада не заработают:

```
cd api
GOOS=js GOARCH=wasm go build -o <каталог>/engine.wasm ./cmd/engine-wasm
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" <каталог>/
```

Затем задайте `ENGINE_DIR=<каталог>` и перезапустите API.

Статическая демонстрационная сборка

Демонстрационная сборка работает без сервера: приложение, браузерное ядро и справочные данные лежат в одном каталоге. Такую сборку можно разместить в любом хранилище статических файлов.

1. Запустите платформу командой `docker compose up -d --wait`.
2. Выполните сценарий сборки:

```
scripts/ci/build-demo.sh
```

Результат появится в каталоге `web/dist`. Демонстрационная сборка ничего не сохраняет, в ней нет учётных записей и идентификаторов запусков.

Что дальше

- Полный список параметров конфигурации приведён в разделе [Переменные окружения](#).
- Порядок проверки платформы описан в разделе [Проверка демонстрационного сценария](#).
- Правила безопасной публикации и резервного копирования описаны в разделе [Сопровождение](#).

Проверка демонстрационного сценария

Этот сценарий позволяет за несколько минут убедиться, что основные функции платформы работают. Сценарий выполняется на проекте «Демо Склад» без входа в систему. Шаг 9 требует учётной записи администратора.

Перед началом

Запустите платформу по инструкции из раздела [Развёртывание](#) и откройте в браузере <http://localhost>.

Шаги проверки

№	Действие	Ожидаемый результат
1	На стартовой странице нажмите Демо Склад	Открыт проект с тремя вариантами, в каждом есть роботы. Карта склада заполнена
2	Откройте вкладку Роботы и выберите робота со статусом «не подходит»	В меню появляется предупреждение. На странице «Итог» показан риск
3	Откройте Расчёт / Итог	Показана таблица трёх сценариев. В таблице есть строка «Зарядных станций»
4	Найдите на вкладке Расчёт раздел «Чувствительность $\pm 20\%$ »	Для каждого варианта показан отдельный блок с изменением цены, объёма и фонда оплаты труда. Пустых строк нет
5	Откройте Симуляцию , выберите вариант 2 и нажмите Запустить симуляцию	Таблица «Проверка симуляцией» обновляется, экономика не пересчитывается. Правка другого варианта не делает эту проверку устаревшей
6	Нажмите Подобрать , затем Взять N в вариант	Число роботов записано в вариант, страница «Итог» пересчитана, проверка симуляцией для варианта действует
7	В симуляции нажмите Сохранить кадр и Показатели CSV	Скачаны файл PNG с кадром и файл CSV с показателями
8	На странице Допущения измените коэффициент загрузки	Размер флота изменился, результат пересчитан
9	Войдите как администратор, откройте Админ / Нормативы , измените ставку и создайте набор допущений	Новый набор использует изменённую ставку

№	Действие	Ожидаемый результат
10	Откройте Экспорт и скачайте файлы PDF и XLSX	В отчётах есть параметры объекта, источники, дата расчёта, оговорка о предварительном характере оценки, зарядные станции и лист «Чувствительность»

Что делать при расхождении

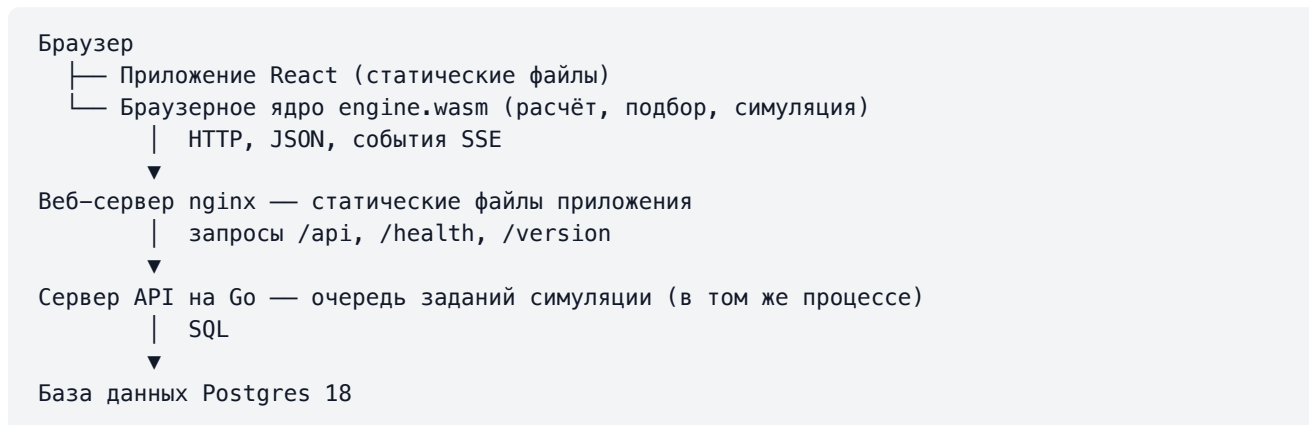
- Если на шаге 1 нет вариантов с роботами или карта пуста, проверьте, что каталог загружён: откройте вкладку **Роботы**, в списке должны быть модели. Если список пуст, выполните сброс данных по инструкции из раздела [Развёртывание](#).
- Если на шаге 5 запуск симуляции не начинается, проверьте, что в проекте нет ошибок карты. Запуск с ошибками карты запрещён.
- Если на шаге 10 отчёт не скачивается с кодом 409, черновик проекта изменился после последнего расчёта. Дождитесь пересчёта на странице «Итог» и повторите выгрузку.

Как это устроено

Архитектура

Платформа состоит из одного исполняемого файла API на Go, статического веб-приложения на React и базы данных Postgres. Расчёт, подбор и симуляция выполняются одним ядром, которое запускается и на сервере, и в браузере.

Компоненты



Компонент	Технологии	Назначение
Сервер API	Go, chi, pgx, sqlc, excelize	Учётные записи, проекты, каталог, расчёт, очередь симуляций, отчёты
Веб-приложение	React, Vite, TypeScript, TanStack Query, Konva	Интерфейс, редактор карты, воспроизведение симуляции
Браузерное ядро	Go, собранный в WebAssembly	Расчёт, проверка карты и симуляция демо без обращения к серверу
База данных	Postgres 18	Проекты, запуски, каталог, сессии, журналы
Веб-сервер	nginx	Раздача приложения и проксирование запросов к API

Очередь заданий симуляции работает внутри процесса API. Отдельные брокеры сообщений и внешние воркеры не нужны.

Единое ядро расчёта

Пакеты `matching`, `econ` и `sim` реализуют подбор, экономику и симуляцию. Сервер и браузер вызывают один и тот же код: сервер собирает его в обычный исполняемый файл, браузер получает файл `engine.wasm` по адресу `/api/engine/`.

Благодаря этому демо-проекты гостя считаются в браузере и дают те же числа, что и сервер. Совпадение проверяется контрольным файлом `contracts/engine/reference.json`.

Интерфейс не содержит собственных формул. Формулы для страницы «Формулы и источники» тоже отдаёт ядро.

Операции над проектом

Данные проекта меняются только операциями: запросом `POST /api/projects/{id}/transactions`. Операция бывает четырёх видов: вставка, установка значения, удаление и перемещение записи. Отдельных запросов `PUT` для содержимого проекта нет.

У такого подхода три следствия:

- несколько редакторов работают над проектом одновременно, а правка, которую сервер не принял из-за конфликта, не теряется, а попадает в список «Не применено» с возможностью восстановить;
- правки можно отменять и повторять в пределах сеанса;
- клиент хранит правки в очереди в браузере и отправляет их при появлении связи.

Расчёт операцией не является: его выполняет сервер по снимку проекта. Подробности см. в [ADR 0003](#).

Модель данных

Основные таблицы Postgres сгруппированы по назначению.

Группа	Таблицы	Содержимое
Пользователи	<code>users</code> , <code>sessions</code> , <code>email_tokens</code> , <code>email_outbox</code> , <code>invitations</code>	Учётные записи, сессии, письма и приглашения
Проекты	<code>projects</code> , <code>project_drafts</code> , <code>project_operations</code> , <code>project_members</code> , <code>project_processes</code> , <code>shared_cost_items</code>	Проект, черновик, журнал операций, участники и роли
Варианты	<code>solution_variants</code> , <code>variant_fleet_items</code> , <code>financing_scenarios</code> , <code>assumption_sets</code>	Состав флота, способы оплаты, наборы допущений

Группа	Таблицы	Содержимое
Запуски	<code>project_versions</code> , <code>calculation_runs</code> , <code>calculation_results</code>	Неизменяемые снимки проекта и результаты расчётов
Симуляция	<code>simulation_jobs</code> , <code>simulation_replications</code> , <code>simulation_artifacts</code>	Очередь заданий, повторы прогнозов, сжатые журналы событий
Каталог	<code>solutions</code> , <code>solution_specs</code> , <code>solution_images</code> , <code>catalog_imports</code> , <code>catalog_exports</code>	Роботы, технические характеристики, изображения, загрузки
Нормативы и журнал	<code>calc_norms</code> , <code>audit_events</code>	Нормативы расчёта, журнал действий

Схема хранится в миграциях `api/migrations` . Миграции применяются автоматически при старте API.

Воспроизводимость результатов

Каждый расчёт и каждая симуляция сохраняются как неизменяемый запуск. Запуск содержит:

- снимок входных данных проекта и его контрольную сумму `input_hash` (SHA-256 канонического JSON);
- контрольную сумму каталога `catalog_content_sha256` и нормативов `norms_sha256` ;
- версии моделей: подбор, экономика (`econ-v3`), симуляция (`sim-v2`);
- зерно генератора случайных чисел (seed);
- уровень подтверждённости: предварительный, настроенный, калиброванный или подтверждённый.

Если контрольная сумма черновика не совпадает с контрольной суммой запуска, результат считается устаревшим: выгрузить его нельзя. Исторический запуск выгружается с пометкой о том, что черновик изменился.

Каталог живой: правка администратора попадает в следующий расчёт. Уже сохранённый запуск остаётся на том каталоге, с которым его посчитали.

Безопасность

- Пароли хранятся в виде хэшей bcrypt. Стоимость хеширования задаёт переменная `BCRYPT_COST` .
- Сессия хранится в cookie `__Host-session` при включённой защите (`COOKIE_SECURE=true`). Срок сессии ограничен временем простоя и предельным сроком.
- Изменяющие запросы проверяются по заголовку `Origin` . Значение должно совпадать с `PUBLIC_ORIGIN` , иначе API отвечает ошибкой `csrf` .

- Доступ к проекту определяется ролью в нём: владелец, редактор или читатель. Проекты разных пользователей изолированы.
- Вкладка «Админ» и запросы `/api/admin/*` доступны только администратору.
- Коммерческие данные гостя в базу и журналы не пишутся.
- Шифрование канала обеспечивает внешний прокси: приложение рассчитано на работу по HTTPS, но в составе `docker-compose.yml` TLS не терминируется.

События в реальном времени

Сервер передаёт события проекта по протоколу SSE (GET `/api/projects/{id}/events`): новые операции, присутствие участников, завершённые запуски, изменение доступа. Благодаря этому участники видят правки друг друга без перезагрузки страницы.

Расширяемость

Платформа рассчитана на добавление новых объектов и решений без переработки ядра:

- схема параметров типа объекта описывается отдельным файлом в `api/internal/objects` (склад, аэропорт, больница);
- каталог загружается таблицей, новые роботы и поля добавляются без изменения кода;
- нормативы расчёта хранятся в базе и правятся в интерфейсе администратора;
- контракты данных описаны схемами JSON в каталоге `contracts/`, программный интерфейс описан файлом `api/openapi.yaml`.

Редактора справочников в интерфейсе нет: новый тип объекта добавляется в код вместе с правилами расчёта.

Структура репозитория

Каталог	Содержимое
<code>api/</code>	Сервер API на Go, миграции, спецификация <code>openapi.yaml</code>
<code>api/cmd/api</code>	Точка входа сервера
<code>api/cmd/engine-wasm</code>	Точка входа браузерного ядра
<code>api/internal/</code>	Пакеты: <code>matching</code> , <code>econ</code> , <code>sim</code> , <code>simbuild</code> , <code>simjobs</code> , <code>ops</code> , <code>exporters</code> , <code>importers</code> , <code>auth</code> , <code>objects</code> и другие
<code>web/</code>	Веб-приложение на React
<code>contracts/</code>	Схемы JSON и эталонные данные ядра
<code>data/</code>	Каталог <code>catalog.csv</code> и технические характеристики <code>seeds/robot_specs.json</code>
<code>docs/</code>	Документация

Каталог	Содержимое
scripts/	Сценарии сборки и проверок

Подбор роботов

Подбор проверяет каждую модель из каталога на соответствие параметрам объекта. Для каждой модели он выдаёт вердикт, оценку и объяснение: какие проверки пройдены, какие нет и каких данных не хватает.

Подбор выполняется на вкладке **Роботы**. Параметры объекта берутся из вкладки **Объект**.

Вердикты

Вердикт	Значение	Что можно сделать
Рекомендован	Все ключевые проверки пройдены, тип объекта совпадает	Выбрать робота в вариант
Требуется проверки	Нет части данных, чтобы подтвердить соответствие: например, в технических характеристиках пусто ключевое поле	Выбрать робота в вариант: расчёт покажет предупреждение
Не подходит	Хотя бы одно жёсткое ограничение нарушено	Добавить в вариант вручную: платформа покажет предупреждение и риск

Платформа не скрывает исключённых роботов. Пользователь может добавить такую модель в сравнение осознанно. Расчёт при этом учитывает риск, а страница «Итог» показывает его.

Проверки

Проверки делятся на жёсткие и мягкие. Нарушение жёсткой проверки исключает робота. Мягкая проверка робота не исключает: она снижает оценку, добавляет замечание и переводит вердикт в «Требуется проверки».

Жёсткие проверки

Проверка	Условие исключения
Проезд	Ширина робота или его минимальный проезд больше проезда на объекте
Грузоподъёмность	Грузоподъёмность меньше массы единицы груза на объекте
Температура	Температура на объекте вне рабочего диапазона робота
Высота потолков	Высота робота больше высоты потолков
Нагрузка на пол	Давление робота на пол больше допустимой нагрузки
Лифт	Масса робота больше грузоподъёмности лифта

Проверка	Условие исключения
Дверные проёмы	Ширина робота больше проёма
Разворот	Круг по диагонали корпуса больше свободного места для разворота
Тип объекта	В карточке робота не указан подходящий тип объекта или его класс не относится к объекту (например, морская техника)

Мягкие проверки

Проверка	Что оценивается
Задачи	У модели нет возможности, которая нужна для выбранной задачи объекта
Тяжёлая логистика	Грузоподъёмность меньше массы паллеты или тележки: модель годится только для лёгкой доставки
Цена	Цена модели не выше бюджета CAPEX из параметров объекта
Полнота данных	Доля заполненных ключевых полей технических характеристик

Если для проверки не хватает данных, платформа не исключает робота, а присваивает вердикт «Требуется проверка» и объясняет, какое поле нужно уточнить.

Оценка

Оценка это число от 0 до 1. Она складывается из четырёх частей с весами:

Часть	Вес	Что учитывается
Соответствие	40%	Запас по габаритам и условиям эксплуатации
Процесс	25%	Совпадение процесса модели с типом объекта
Полнота данных	20%	Доля заполненных ключевых характеристик
Ценовой пояс	15%	Цена относительно бюджета CAPEX

Оценка помогает упорядочить роботов внутри одного вердикта. Части оценки и веса отображаются в разборе робота.

Порядок в списке

Порядок списка определяется двумя правилами.

1. Сначала идут роботы с вердиктом «Рекомендован», затем «Требуется проверки», затем «Не подходит».
2. Внутри одного вердикта роботы, для которых рассчитана окупаемость, идут первыми, по возрастанию срока окупаемости. Остальные располагаются по оценке.

Окупаемость для списка рассчитывается для случая «этот робот один в варианте». Кнопка **Выбрать** записывает в вариант именно этот случай, поэтому предложение на странице «Роботы» совпадает с итогом расчёта.

Как читать объяснение

Для каждого робота платформа показывает разбор:

- **Причины.** Короткий список того, почему вердикт именно такой.
- **Жёсткие проверки.** Что пройдено и что нарушено, со значениями на объекте и у робота.
- **Мягкие замечания.** Что снижает оценку.
- **Недостающие данные.** Какие поля нужно уточнить в каталоге.
- **Источник.** Ссылка на источник характеристики и уровень подтверждённости.

Такой разбор позволяет понять, какое именно условие исключило модель, и решить, что менять: объект, каталог или выбранную модель.

Экономическая модель

Модель считает, во что обойдётся внедрение роботов и когда оно окупится. Она строит годовой денежный поток и сравнивает три сценария по одним и тем же показателям. Формулы на странице «Формулы и источники» выдаёт то же ядро, которое считает, поэтому текст на странице всегда совпадает с расчётом.

Сценарии

Сценарий	Что моделируется
База	Текущий процесс без роботов. Затраты на персонал берутся из параметров объекта
Покупка	Оборудование покупается. Затраты делятся на разовые (CAPEX) и ежегодные (OPEX)
RaaS	Оборудование арендуется по модели «роботы как услуга» (Robot as a Service). Платёж за использование идёт весь горизонт, оборудование не покупается

Что рассчитывается

Для каждого сценария платформа показывает:

- количество роботов и зарядных станций;
- CAPEX по статьям;
- годовой OPEX и его изменение относительно базы;
- годовой денежный эффект;
- простую окупаемость, ROI и затраты за горизонт (TCO);
- чистую приведённую стоимость (NPV), внутреннюю норму доходности (IRR) и дисконтированную окупаемость, справочно;
- денежный поток по годам.

Горизонт расчёта для склада составляет от 5 до 10 лет. Главная метрика: простая окупаемость.

Формулы

Все суммы считаются в постоянных ценах базового года. Вложения относятся к году 0, платежи и эффекты к концу года.

Количество роботов и зарядных станций

$$\text{флот} = \text{пик} / (\text{производительность} * \text{доступность} * \text{загрузка}), \text{вверх до целого};$$
$$\text{затем} * (1 + \text{резерв}), \text{вверх до целого}$$

Зарядные станции считаются по времени работы и зарядки из технических характеристик:

станции = количество * время зарядки / (время работы + время зарядки), вверх до целого

Если характеристик нет, берётся одна станция на четырёх роботов. У флота не меньше одной станции. Стоимость станций входит в инфраструктуру. Если на карте станций меньше, чем нужно, расчёт показывает риск.

CAPEX

CAPEX покупки = (оборудование + доставка + инфраструктура + ПО + интеграция + пусконаладка + обучение) * (1 + резерв на непредвиденное)

CAPEX RaaS = (инфраструктура + ПО + интеграция + пусконаладка + обучение) * (1 + резерв на непредвиденное)

Доставка составляет 3% стоимости оборудования и входит только в CAPEX покупки. В RaaS оборудования и доставки нет.

OPEX

OPEX покупки = ФОТ остатка + сервис и ремонт + лицензии + энергия + расходные материалы + связь + персонал эксплуатации

OPEX RaaS = ФОТ остатка + плата по тарифу + энергия + расходные материалы + связь + персонал эксплуатации

- **ФОТ остатка.** Затраты на персонал, который остаётся после внедрения.
- **Персонал эксплуатации.** Один техник на 10 роботов в каждую смену: вверх до целого (флот / 10) * число смен * зарплата * 12 * (1 + начисления) .
- **ФОТ.** штат * оклад * 12 * (1 + начисления) . Взносы с части дохода выше предельной базы 2 979 000 ₽ на человека в год считаются по ставке 15,1%.

Денежный эффект и окупаемость

денежный эффект = OPEX базы – OPEX сценария

поток года t = денежный эффект – замена батарей – повторная покупка

год 0 = –CAPEX

Простая окупаемость: момент, когда накопленный денежный поток достигает нуля. Без замен она равна CAPEX / денежный эффект . Расписание рассчитывается на 30 лет. Окупаемость, которая наступает позже горизонта, не экстраполируется, её ищут внутри этих 30 лет.

Справочные показатели:

NPV = –CAPEX + сумма по годам t от 1 до горизонта: поток года t / $(1 + r)^t$

IRR = ставка r , при которой NPV равен нулю

ROI = (сумма потоков за горизонт – CAPEX) / CAPEX * 100%

TCO = CAPEX + OPEX за год * горизонт + замены батарей + повторные покупки

Ставка дисконтирования r по умолчанию равна 15% в год. На простую окупаемость она не влияет.

Замены и амортизация

- Робот покупается заново, когда срок его службы заканчивается внутри горизонта. Это относится только к покупке.
- Замена батареи стоит 15% цены робота и выполняется на 4-м и 8-м году и далее, пока год лежит внутри срока службы и горизонта. Это разовый платёж в своём году, в годовой ОРЕХ он не входит.
- Амортизация линейная. Цена робота делится на срок его службы, а доставка, инфраструктура, ПО, интеграция и пусконаладка делятся на срок службы по умолчанию (7 лет). Обучение и резерв не амортизируются.
- Амортизация нужна для бухгалтерского эффекта (денежный эффект минус амортизация) и в денежный поток не входит.

RaaS

Плата RaaS бывает фиксированной, переменной или смешанной:

фиксированная = доля цены в месяц * 12 * цена * флот
переменная = ставка за операцию * годовой объём
смешанная = доля фиксированной части * фиксированная
+ (1 - доля) * переменная

Если своя ставка за операцию не задана, берётся ставка, выведенная из фиксированной платы при плановом объёме. Срок контракта равен горизонту. Доставка, сервис, лицензии и батареи входят в плату.

НДС

Ставка НДС по умолчанию 22% (с 1 января 2026 года). Одна ставка применяется ко всем строкам, которые несут НДС: оборудованию, инфраструктуре, лицензиям, энергии, расходным материалам и общим затратам. Зарплата НДС не облагается. Если НДС принимается к вычету, в денежный расчёт идёт цена без НДС: $\text{цена с НДС} / (1 + \text{ставка})$.

Нормативы по умолчанию

Норматив это число, которое модель берёт, когда значение не задано ни в проекте, ни в карточке робота. Администратор меняет нормативы на вкладке **Админ / Нормативы**. Пустое поле набора допущений принимает значение норматива.

Норматив	Значение по умолчанию
Доступность	90%
Загрузка	80%
Резерв флота	15%
Инфраструктура	15% стоимости оборудования

Норматив	Значение по умолчанию
Программное обеспечение	8% стоимости оборудования
Интеграция (есть или нет готовых интерфейсов)	12% или 20% стоимости оборудования
Пусконаладка	5% стоимости оборудования
Обучение	3% стоимости оборудования
Резерв на непредвиденное	10% суммы статей CAPEX
Доставка	3% стоимости оборудования
Мощность робота	2,0 кВт
Стоимость электроэнергии	6,5 ₽/кВт·ч
Лицензия	80 000 ₽ на работа в год
Расходные материалы	20 000 ₽ на работа в год
Связь	12 000 ₽ на работа в год
Роботов на одного техника	10
Зарплата техника	90 000 ₽ в месяц, без НДС
Сервис и ремонт	8% стоимости оборудования в год
Срок службы	7 лет
Батарея	15% цены робота каждые 4 года
Плата RaaS	2,5% цены робота в месяц
Доля фиксированной части в смешанной плате	50%
Ставка НДС	22%
Ставка дисконтирования	15%

Загрузка 80% взята из середины диапазона 70-85% для роботов класса AMR из легенды датасета организаторов.

Чувствительность

Для каждого варианта с флотом платформа показывает, как меняется результат при изменении на 20% в обе стороны:

- цены оборудования;
- объема операций;
- фонда оплаты труда.

При изменении объема пересчитывается и размер флота: количество каждой позиции умножается на тот же коэффициент и округляется вверх. База ФОТ не меняется.

Что не учитывается

Модель не считает:

- налог на прибыль и налоговый щит от амортизации;
- инфляцию и рост зарплат;
- постепенный ввод в эксплуатацию: экономия начинается с года 1;
- остаточную стоимость оборудования на конец горизонта;
- заёмное финансирование (кредит, лизинг): расчёт идёт на собственные средства.

Из-за того, что остаточная стоимость не учитывается, NPV, ROI и затраты за горизонт получаются консервативными. Особенно это заметно, когда работа пришлось купить снова незадолго до конца горизонта.

Интерпретация результата

Платформа не делает вывод по единственному порогу. Она показывает число, риски и пояснение. Срок окупаемости попадает в один из интервалов: до 3 лет, от 3 до 5 лет, более 5 лет, либо окупаемость не наступает в пределах расчёта.

К результату всегда прилагается оговорка: оценка предварительная, перед внедрением нужно обследование площадки.

Источники допущений

Допущения расчёта показаны на странице «Формулы и источники» и в отчёте. Источник нормативов: вкладка «Нормативы». Источник технических характеристик: карточка робота со ссылкой на страницу производителя или другой источник.

Три допущения выбрала команда разработки, а не заказчик: ставка дисконтирования 15%, предельная база взносов 2026 года и тариф RaaS. Обоснование приведено в [ADR 0004](#).

Как читать результат

Раздел объясняет, как устроены страница расчёта и отчёты и когда результату нельзя доверять. Формулы и нормативы описаны в разделе [Экономическая модель](#), симуляция в разделе [Симуляция](#).

Три сценария

- **База.** Текущие затраты труда, роботов нет.
- **Покупка.** Оборудование приобретается и обслуживается силами предприятия.
- **RaaS.** Предприятие платит за использование роботов, оборудование не покупается.

Сценарии считаются для складов, аэропортов и больниц. Событийная симуляция по плану есть только у склада. У аэропорта и больницы платформа выполняет экономический расчёт и быструю проверку без карты.

Страница расчёта

Вкладка **Расчёт** состоит из страниц.

Страница	Что показывает
Итог	Таблицу сценариев (CAPEX, OPEX, эффект, окупаемость, ROI), состав флота, проверку симуляцией, чувствительность
Детализация	Статьи CAPEX и OPEX по каждому сценарию
Симуляция	Запуск симуляции на карте склада и её результаты
Допущения	Наборы допущений и ключевые параметры, которые можно менять
Формулы и источники	Формулы модели, значения нормативов, источники и допущения
Экспорт	Выгрузка отчётов PDF и XLSX
История	Список сохранённых запусков

Страницы «История» в демо нет. Страница расчёта пересчитывается сама после каждой правки.

Отчёты PDF и XLSX

Отчёты короткие и рассчитаны на человека, который выбирает вариант. В них есть:

- параметры объекта и выбранные роботы;
- сравнение базы и вариантов: первоначальные вложения, ежегодные затраты, годовой денежный эффект, простая окупаемость и затраты за горизонт;
- состав лучшего варианта и допущения, которые могут изменить вывод;

- источники данных, дата расчёта и оговорка о том, что оценка предварительная.

Лучшим считается вариант с самой короткой простой окупаемостью в пределах горизонта. Если окупаемость у нескольких вариантов одинакова или длиннее горизонта, отчёт не выделяет ни один вариант и пишет об этом.

Чистая приведённая стоимость, внутренняя норма доходности, дисконтированная окупаемость и денежный поток по годам видны на странице расчёта, но в отчёты не входят.

Отчёт строится из снимка запуска. Сам снимок (входные данные, контрольные суммы, карта) в файл не попадает и остаётся в базе данных.

Когда результату нельзя верить

Экономика и симуляция расходятся

Если симуляция и экономический расчёт расходятся больше чем на 15%, у результата появляется предупреждение. Платформа его не скрывает. Найдите вход, из-за которого возникло расхождение, и устранили его. Пока расхождение есть, не передавайте отчёт как согласованный.

Черновик изменён после расчёта

После правки черновика текущий результат относится уже к другим входным данным, поэтому выгрузить его нельзя. Дождитесь пересчёта и повторите выгрузку. Исторический запуск выгружается с пометкой, что черновик с тех пор изменился.

Не хватает данных

Если у выбранного робота не заполнены ключевые технические характеристики, расчёт берёт значения из нормативов и показывает предупреждение. Список таких допущений виден на странице **Формулы и источники**. Уточните характеристики в каталоге и пересчитайте результат.

Основные допущения

- Цены постоянные, взяты для базового года. Инфляции и роста зарплат нет.
- Вложения относятся к году 0, платежи к концу года.
- Расписание рассчитывается на 30 лет. Окупаемость длиннее горизонта не экстраполируется.
- Остаточная стоимость оборудования не учитывается.
- Налог на прибыль и финансирование не моделируются.
- В RaaS доставка, сервис, лицензии и батареи входят в плату. Тариф задан командой проекта, а не договором.

Полный перечень допущений приведён на странице **Расчёт / Формулы и источники**.

Симуляция

Симуляция имитирует работу роботов на плане склада. Она проверяет, что производительность флота из экономического расчёта достижима на конкретной карте, и показывает загрузку оборудования, простои и узкие места.

Симуляция на карте есть только у склада. Для аэропорта и больницы платформа выполняет быструю проверку без карты по каждой позиции флота.

Как работает модель

Модель событийная: время идёт от события к событию, а не равными шагами.

Задания и спрос

Задание состоит в том, чтобы перевезти груз от точки забора к точке доставки. Точки берутся из потоков на карте. Спрос в час считается так:

$$\text{спрос в час} = \text{спрос в сутки} / \text{единиц в задании} / \text{активные часы суток} * \text{профиль часа}$$

Активные часы определяются окном смены.

Роботы

Профили роботов различаются действиями (подъём под грузом или вилы с подъёмом на ярус), габаритами, скоростью с грузом и потреблением энергии. Значения из каталога переопределяют значения класса. Подставленные значения перечислены в допущениях запуска.

Робот класса AMR грузоподъёмностью от 400 кг возит паллеты по полу, но не ставит их в стеллаж. Размещение и отгрузку из ячеек выполняет паллетный робот.

Общие ресурсы

Роботы делят между собой док, узкий участок и зарядную станцию. К каждому ресурсу выстраивается очередь по порядку прихода.

Узкий участок: однополосный отрезок маршрута, у которого ширина меньше $2 * \text{ширина} + 1,5 * \text{зазор}$ самого широкого класса роботов. Цепочка таких отрезков без развилок работает как ресурс вместимостью 1. Робот, остановившийся в тупике участка, удерживает его.

Маршрут идёт в обход участков, которые уже, чем $\text{ширина} + \text{зазор}$ робота.

Диспетчеризация

Задания назначаются по одному из трёх правил:

- по порядку поступления;
- ближайшему свободному роботу;
- по приоритету соблюдения SLA.

SLA оценивается по времени ожидания до назначения и по полному циклу задания.

Режимы прогона

Режим	Как работает	Что показывает
Детерминированный	Один прогон	Одно значение каждого показателя
Вероятностный	До 30 повторов: пуассоновский поток заданий, логнормальные длительности	Медиану, минимум, максимум, 10-й и 90-й процентиля

SLA считается выполненным, если 90-й перцентиль доли нарушений не выше допуска. Допуск по умолчанию 5%.

Узкие места

Платформа ищет узкое место в два шага:

1. Сначала проверяет ресурсы. Ресурс насыщен, если его загрузка не меньше 85% или средняя очередь не меньше половины вместимости.
2. Если насыщенных ресурсов нет, проверяет флот. Флот считается узким местом, если роботы свободны меньше 10% времени.

Сверка с экономикой

После прогона платформа сравнивает производительность из симуляции с той, что заложена в экономике. Если расхождение больше 15%, результат помечается предупреждением. Оно остаётся, пока причина не устранена.

Сверка привязана к варианту. Правка одного варианта не делает устаревшей сверку другого.

Подбор числа роботов

На странице **Симуляция** кнопка **Подобрать** ищет наименьшее число роботов, при котором покрытие спроса не меньше 85% и SLA укладывается в допуск. Поиск идёт прогонами симуляции.

Кнопка **Взять N в вариант** записывает найденное число в вариант одной операцией. После этого страница «Итог» пересчитывается, а сверка для этого варианта считается действующей.

Карта склада

На карте заданы зоны, проезды, препятствия, точки забора и доставки, зарядные станции и потоки. Карту можно нарисовать в редакторе (**Объект / Карта**) или загрузить из файла плана.

- Координаты хранятся в пикселях страницы, метры получаются умножением на масштаб. Исходный файл плана на сервер не отправляется.

- Черновик карты сохраняется даже с ошибками, ошибки записываются в документ. Запуск симуляции с ошибками карты запрещён.
- Если у проекта нет своей карты, используется шаблон склада 84 x 52 м. Тогда результат получает уровень подтверждённости «предварительный» и предупреждение. С картой проекта уровень «настроенный».

Запуск симуляции

Симуляция проекта выполняется заданием в очереди. Задания обрабатывает тот же процесс API. Пользователь видит статус задания и может отменить его или повторить.

- Не больше трёх активных заданий на пользователя и одного на проект.
- Задание, которое 30 секунд не обновляло статус, возвращается в очередь. Если попытки закончились, оно завершается с ошибкой.
- Предел времени одного задания по умолчанию 10 минут. Значения настраиваются, см. [Переменные окружения](#).

Демо считают симуляцию в браузере. Результаты хранятся там же: пять последних запусков на каждое демо.

Воспроизведение

Элемент	Назначение
Пуск, Пауза, Сначала	Управление воспроизведением
Скорость	От 1x до 600x
Выбор варианта	Какой вариант показывать
Сохранить кадр	Сохраняет текущий кадр в файл PNG
Показатели CSV	Сохраняет показатели запуска в файл CSV

Воспроизводится один повтор, ближайший к медиане нарушений и производительности. Его журнал хранится в сжатом виде 90 дней, закреплённый журнал не удаляется. В журнал попадает не больше 400 000 событий, остальные отбрасываются, и запуск получает пометку об усечении.

Что не моделируется

- Автоматическое построение графа маршрутов, встречное движение внутри двухполосного участка, лифты и двери.
- Отказы и обслуживание роботов: данных для них нет, модель работает без отказов.

Подробности реализации приведены в [ADR 0002](#).

Данные и каталог

Платформа работает на заранее загруженных данных: каталоге роботов, технических характеристиках (ТТХ) и справочных значениях. Внешние источники во время работы не запрашиваются, поэтому недоступность сайта производителя не останавливает демонстрацию.

Источники данных

Файл	Что содержит	Как используется
<code>catalog_export_v4.csv</code>	Названия, производители, цены, отрасли и сценарии	Копия лежит в <code>data/catalog.csv</code> , загружается при первом запуске
Примеры решений организаторов (DOCX)	Примеры ТТХ для части роботов	Исходные значения в <code>data/seeds/robot_specs.json</code>
Параметры объектов (XLSX)	Демонстрационные параметры склада, аэропорта и больницы	Основа демо-проектов и шаблона загрузки параметров
Открытые страницы производителей	Дополнительные ТТХ	Добавлены в <code>data/seeds/robot_specs.json</code> со ссылкой на источник и датой

Каталог содержит 223 строки, в них 187 различных идентификаторов.

Загрузка каталога при старте

При старте API таблица решений заполняется из `data/catalog.csv`, только если она пуста. Повторный запуск файл заново не загружает и строки не удаляет.

Технические характеристики из `data/seeds/robot_specs.json` применяются при каждом старте: существующая строка дополняется, решения не удаляются. Посев только заполняет пустые поля и добавляет недостающих роботов. Он не перезаписывает уже сохранённое значение и пропускает роботов, которых правил администратор. Поэтому новые значения из файла полностью видны только на чистой базе (`docker compose down -v`), а на работающей базе меняются через редактор администратора.

Повторяющиеся идентификаторы

В файле каталога 23 идентификатора встречаются несколько раз. Это одно изделие для разных отраслей, сценариев или цен, а не второй робот. Организаторы просили не удалять и не объединять такие строки, поэтому каждая строка стала отдельным решением.

- Первая строка идентификатора сохраняет исходный идентификатор, поэтому сохранённые флоты и запуски остаются действительными.
- Остальные строки получают производный идентификатор, который не зависит от порядка строк в файле.

- У каждой строки остаются свои отрасль, сценарий, случаи применения и цена.
- Технические характеристики применяются ко всем строкам одного идентификатора.

Подробности и таблица расхождений цен приведены в `data/seeds/NOTES.md`.

Технические характеристики

В файле характеристик 58 роботов. У 19 значения взяты из документа организаторов, остальные добавлены по открытым страницам производителей. ТТХ найдены примерно у 55 из 187 роботов каталога. У остальных поля пусты и показываются как «Нет в ТТХ».

Правила внесения значений:

- значение вносится, только если открытая страница называет его для этой модели, а ссылка на страницу записана в источник;
- первой берётся страница производителя, затем страница дистрибьютора, отраслевого портала или пресс-релиз, если у производителя нет читаемой страницы;
- значение, по которому источники расходятся, не вносится;
- пробел объясняется в файле `data/seeds/NOTES.md`;
- описания пересказаны по-русски, до 300 символов, фотографии не копировались.

Не найдены ни на одной читаемой странице: радиус разворота (ни у одного робота), время зарядки у Ronavi и MOROS, габариты, масса и минимальный проезд у DMR Carrier P.

Срок службы и годовая доля сервиса не найдены ни в одном источнике. Расчёт подставляет допущение и показывает это на странице «Формулы и источники».

Происхождение данных

У каждого значения технических характеристик хранятся:

- ссылка на источник;
- дата получения;
- уровень подтверждённости: данные производителя или измерения;
- примечание, если значение вызывает сомнения.

В разборе подбора и в карточке робота эти сведения показываются рядом со значением.

Параметры объекта

Параметры объекта вводятся формами на вкладке **Объект** или загружаются из файла. Шаблон XLSX доступен на вкладке объекта. Загрузка принимает XLSX и CSV. Платформа проверяет типы, единицы, обязательные поля и диапазоны значений и показывает ошибки до применения.

Значение «Неизвестно» записывается пустым полем. Пустое поле не считается значением: подбор и расчёт отмечают недостающие данные.

Обновление каталога

Администратор обновляет каталог на вкладке **Админ / Каталог**: правит работа в оверлее или загружает таблицу. Порядок работы описан в [руководстве администратора](#).

Правка работа попадает в следующий расчёт. Уже сохранённый запуск остаётся на каталоге, с которым его посчитали, потому что запуск записывает контрольную сумму каталога.

Руководства

Руководство пользователя

Руководство описывает работу с платформой по задачам. Каждая задача записана как последовательность шагов.

Общие правила интерфейса

- **Правки сохраняются сами.** Кнопки «Сохранить» нет. Поле сохраняется через 0,6 секунды после последней клавиши, по Enter и при выходе из поля. Списки, флажки и переключатели сохраняются сразу.
- **Ошибка в поле.** Значение, которое не прошло проверку, остаётся в поле вместе с сообщением и не отправляется. Пока поле не исправлено, действует прежнее значение.
- **Проблемы с сохранением.** Если нет сети, сессия закончилась или правки отклонены, на кнопке учётной записи появляется точка, а в меню написана причина. Правки ждут в браузере и не пропадают при перезагрузке страницы.
- **Отмена и повтор.** Cmd+Z и Cmd+Shift+Z на macOS, Ctrl+Z и Ctrl+Shift+Z (или Ctrl+Y) в остальных системах. Отменяются правки текущей вкладки в проекте.
- **Поиск.** Cmd+K на macOS или Ctrl+K в остальных системах. Поиск находит вкладки, параметры со значениями, процессы, объекты карты, наборы допущений, формулы, риски, запуски, другие проекты и до пяти роботов из каталога.
- **Пустое поле.** Показывает «нет данных». Пустое поле набора допущений, которое перекрывает норматив, показывает текущее значение норматива серым цветом. Подсказка «?» рядом с таким полем говорит «по нормативу». Это не то же самое, что «нет данных».
- **«Неизвестно».** Поле, которое допускает такое значение, оставляют пустым. Подсказка «?» сообщает об этом.
- **Подсказки.** Значок «?» после названия поля содержит полное название, примечание и допустимый диапазон. Единица измерения стоит внутри поля.
- **Длинные формы.** Разбиты на подвкладки. Подкладка с ошибкой помечена, поэтому ошибку не спрятать на вкладке, которую никто не открывает.
- **Панель действий.** Полоса у нижнего края окна. Слева состояние страницы, если есть о чём сообщить (поля с ошибками, устаревший результат, что нашла проверка карты). Справа действия страницы, главное первым. Ниже 1280 px второстепенные действия сворачиваются в меню «Действия».
- **Обновление платформы.** Новая версия устанавливается сразу. Вкладка со старой версией перезагружается сама, когда пользователь переходит на другую вкладку или подкладку, но не при вводе в поле и не при открытом диалоге.
- **Ошибки.** Страница ошибки говорит, что случилось, и предлагает одно действие. Если сервер не отвечает, интерфейс показывает сообщение «Сервер не отвечает. Повторите через минуту.»

Вход и гостевой режим

Чтобы открыть демо без входа, нажмите **Демо Склад**, **Демо Аэропорт** или **Демо Больница** на стартовой странице. Правки демо хранятся в браузере и не попадают на сервер.

Чтобы создать проект, войдите или зарегистрируйтесь. Демонстрационные учётные записи описаны в разделе [Развёртывание](#).

Создать проект

1. На стартовой странице нажмите **Новый проект**.
2. Введите название и выберите тип объекта: склад, аэропорт или больница.
3. Подтвердите создание.

Тип объекта выбирается один раз при создании. Изменить его позже нельзя.

Проекты на стартовой странице показаны карточками. Карточка содержит действия **Экспорт**, **Копия** и **Удалить**. Удалённый проект попадает в **Корзину** и хранится там 30 дней (срок задаёт администратор платформы, см. [Сопровождение](#)). Из корзины проект можно восстановить.

Заполнить параметры объекта

1. Откройте вкладку **Объект** и заполните подвкладки. Набор подкладок зависит от типа объекта.
2. На подкладке **Процессы** опишите процессы объекта. Карточка процесса открывается в диалоге, кнопка **Готово** закрывает его.
3. Для склада на подкладке **Карта** нарисуйте план или загрузите файл плана.

Чтобы загрузить параметры из файла, скачайте шаблон XLSX на вкладке объекта, заполните его и загрузите обратно. Платформа проверит значения и покажет ошибки. Формат CSV тоже принимается.

Подобрать робота

1. Откройте вкладку **Роботы**. Вверху показано предложение с причинами, по которым оно первое. Ниже идут варианты (Вариант 1, 2, 3), а под ними все роботы каталога.
2. Роботы упорядочены по вердикту, а внутри вердикта по окупаемости. Количество, CAPEX и окупаемость рядом с роботом посчитаны для случая, когда он один в варианте.
3. Нажмите **Выбрать** у нужного робота, чтобы записать этот случай в вариант. Страница «Итог» покажет те же числа, что и список.
4. Название робота открывает его карточку с описанием, характеристиками и объяснением подбора.

Название варианта меняется на месте. Если подбор исключил робота, его всё равно можно взять в вариант: меню покажет предупреждение, а страница «Итог» покажет риск. См. [Подбор роботов](#).

Список фильтруется по группе, статусу, компании, типу объекта и характеристикам. Кнопка **Сбросить фильтры** возвращает полный список. Роботов можно отметить и сравнить рядом.

Посчитать экономику

Откройте вкладку **Расчёт**. Страница пересчитывается сама после каждой правки.

- **Итог.** Таблица трёх сценариев, состав флота, проверка симуляцией, чувствительность.
- **Детализация.** Статьи CAPEX и OPEX.
- **Допущения.** Значения, которые можно менять: стоимость персонала, режим работы, производительность, стоимость оборудования и обслуживания, коэффициент загрузки, горизонт.
- **Формулы и источники.** Формулы, нормативы и допущения.

Чтобы увидеть, как результат зависит от допущения, измените его на странице **Допущения**. Поля, от которых зависит серверный расчёт, сохраняются только по Enter или при выходе из поля.

Как читать числа, описано в разделе [Как читать результат](#).

Проверить симуляцией

1. Откройте **Расчёт / Симуляция** и выберите вариант.
2. Нажмите **Запустить симуляцию** и дождитесь завершения. Статус задания виден на странице, задание можно отменить.
3. Чтобы подобрать число роботов, нажмите **Подобрать**, затем **Взять N в вариант**.
4. Для просмотра используйте **Пуск**, **Пауза**, **Сначала** и выбор скорости.
5. Кнопка **Сохранить кадр** сохраняет PNG, кнопка **Показатели CSV** сохраняет таблицу показателей.

См. [Симуляция](#).

Выгрузить отчёт

1. Откройте **Расчёт / Экспорт**.
2. Скачайте файл PDF для руководителя или XLSX для расчётов.

Отчёт содержит оговорку: результат предварительный и требует обследования объекта. Если черновик изменился после расчёта, текущий результат выгрузить нельзя. Дождитесь пересчёта. Прежние запуски выгружаются со страницы **История**.

Совместная работа

Владелец проекта приглашает других пользователей и назначает роли: редактор или читатель. Читатель проект не меняет. Пока несколько человек работают в одном проекте, на панели видны их лица и место, где они работают. Правка, которую сервер не принял из-за конфликта, попадает в список «Не применено». Оттуда её можно восстановить.

Руководство администратора

Вкладка **Админ** видна только администратору. Она состоит из подвкладок **Каталог**, **Нормативы**, **Приглашения** и **Журнал**. Роли и права описаны в разделе [Роли и доступ](#).

Каталог

Править одного робота

1. Откройте **Админ / Каталог** и нажмите на название робота.
2. Измените поля в оверлее. Поля с источником показывают ссылку, дату и уровень подтверждённости.
3. Закройте оверлей. Правка сохранена и попадёт в следующий расчёт.

Робота можно дублировать и отправить в архив. Архив убирает робота из обычного списка, строка остаётся в базе. Кнопка **Новое решение** создаёт пустую карточку.

Значения, которые правил администратор, посев ТТХ при перезапуске не перезаписывает.

Загрузить таблицу

Для загрузки есть два образца: образец на одного робота (поле в строке) и образец таблицы (робот в строке). Выгрузка каталога тоже подходит как файл для загрузки.

1. Нажмите загрузку и выберите файл.
2. Проверьте предпросмотр. Строка с ошибкой в поле не применяется, ошибки можно выгрузить списком.
3. Подтвердите загрузку.

Правила загрузки:

- Строка с тем же идентификатором обновляет существующее решение, а не создаёт второе.
- Если файл сделан из выгрузки каталога, предпросмотр сравнивает три стороны: значение в файле, значение в каталоге и значение в выгрузке, на основе которой сделан файл. Поле, изменённое только в файле, берётся из файла. Поле, изменённое только в каталоге, остаётся как в каталоге. Поле, изменённое по-разному в обоих местах, считается конфликтом. По умолчанию в конфликте побеждает каталог, пока администратор не выберет файл.
- Если файла-основания нет, пустая ячейка не затирает сохранённое значение.
- Пока предпросмотр открыт, каталог мог измениться. В таком случае подтверждение отклоняется, и предпросмотр нужно открыть заново.
- Роботов, которых нет в файле, загрузка не удаляет. Их можно отправить в архив вручную.
- Ссылка на фото в файле забирается после сохранения строки.

Нормативы

На подвкладке **Нормативы** заданы доли и ставки модели: доступность, загрузка, резерв, доли статей CAPEX, стоимость энергии, тариф RaaS, ставка НДС, ставка дисконтирования и другие. Список значений по умолчанию приведён в разделе [Экономическая модель](#).

Пустые поля наборов допущений берут эти числа. Ставка НДС и ставка дисконтирования применяются и к новым наборам. Запись «Нормативы изменены» попадает в журнал.

Чтобы изменить норматив:

1. Откройте **Админ / Нормативы**.
2. Измените значение и подтвердите правку.
3. Откройте проект и убедитесь, что расчёт пересчитан. Уже сохранённые запуски остаются на прежних нормативах: запуск записывает контрольную сумму нормативов.

Приглашения

Если включён режим приглашений (`REGISTRATION_MODE=invite`), новая учётная запись создаётся только по приглашению.

1. Откройте **Админ / Приглашения** и введите адрес электронной почты.
2. При необходимости привяжите приглашение к проекту и роли.
3. Отправьте приглашение. Получатель получит письмо со ссылкой.

В режиме `MAIL_MODE=capture` письма наружу не уходят и остаются в базе. Как включить отправку, описано в разделе [Сопровождение](#).

Журнал

Журнал записывает действия с учётными записями, участниками, проектами, каталогом, приглашениями и нормативами: кто, что и когда сделал. Правки полей проекта в журнал не попадают, они хранятся как операции проекта. Записи хранятся 365 дней.

Роли и доступ

Платформа различает роли на уровне платформы и роли внутри проекта.

Роли платформы

Роль	Что доступно
Гость	Каталог и три демо-проекта. Демо живут в браузере. Создать свой проект гость не может. Расчёт и выгрузка PDF и XLSX идут через API и не пишут коммерческие данные в базу и в журналы
Пользователь	Свои проекты: создание, правка, копирование, удаление, расчёт, симуляция, экспорт
Администратор	Всё, что у пользователя, и вкладка Админ : каталог, нормативы, приглашения, журнал

Демонстрационные учётные записи создаются при старте API, если записей с такими адресами ещё нет. Адреса и пароли приведены в разделе [Развёртывание](#). Перед любой публикацией в сети пароли нужно сменить, см. [Сопровождение](#).

Роли в проекте

Роль	Что доступно
Владелец	Всё, включая управление участниками и удаление проекта
Редактор	Правка параметров, процессов, вариантов и карты, запуск расчётов и симуляций
Читатель	Просмотр. Проект читатель не меняет

Проекты разных пользователей изолированы: пользователь видит только проекты, где он участник.

Регистрация

По умолчанию регистрация открыта. Если включён режим приглашений, новая учётная запись создаётся только по приглашению с вкладки **Админ / Приглашения**. Как включить режим, написано в разделе [Сопровождение](#).

Что делает каталог при старте

Файл `data/catalog.csv` загружается, только если таблица решений пуста. Повторный старт файл заново не загружает и строки не удаляет.

Технические характеристики из `data/seeds/robot_specs.json` применяются при каждом старте: существующая строка дополняется, решения не удаляются.

Если в файле каталога один идентификатор встречается дважды, первая строка сохраняет идентификатор, а остальные становятся отдельными решениями с производным идентификатором. Подробности приведены в разделе [Данные и каталог](#).

Правка робота в разделе **Админ** попадает в следующий расчёт. Сохранённый запуск остаётся на каталоге, с которым его посчитали.

Сопровождение

Раздел описывает то, что нужно сделать перед публикацией платформы, и то, что может сломать стенд или данные. Команды первого запуска приведены в разделе [Развёртывание](#), полный список переменных окружения в разделе [Переменные окружения](#).

Перед публикацией в сети

1. Смените демонстрационные пароли (`DEMO_USER_PASSWORD` , `DEMO_ADMIN_PASSWORD`).
2. Настройте HTTPS на внешнем прокси. В составе `docker-compose.yml` шифрование не включено.
3. Задайте `COOKIE_SECURE=true` .
4. Задайте `PUBLIC_ORIGIN` равным адресу сайта, с тем же протоколом и портом, с которыми открывается интерфейс. Если значение не совпадает с адресом страницы, изменяющие запросы отклоняются с ошибкой `csrf` .
5. Не оставляйте `APP_ENV=local` . При этом значении открыт просмотр писем `GET /api/dev/outbox` .
6. Выберите режим регистрации: `REGISTRATION_MODE=invite` разрешает регистрацию только по приглашению.
7. Решите, как отправлять письма. По умолчанию (`MAIL_MODE=capture`) письма остаются в базе и наружу не уходят. Чтобы отправлять их, задайте `MAIL_MODE=smtp` и параметры `SMTP_HOST` , `SMTP_PORT` , `SMTP_USER` , `SMTP_PASSWORD` .

Запуск и сброс

Запуск:

```
docker compose up --build
```

Интерфейс открывается по адресу `http://localhost`, проверка API по адресу `http://localhost:8080/health`. Миграции применяются при старте API.

Сброс удаляет том базы. Пропадают проекты, запуски и учётные записи:

```
docker compose down -v
```

Следующий запуск заново загрузит каталог из CSV, потому что таблица пуста, и создаст демонстрационные учётные записи. На стенде, данные которого нужно сохранить, `-v` не указывайте.

Резервная копия базы

Перед миграцией снимите копию. В локальном Compose сервис называется `db` , пользователь и база `app` . На стенде подставьте свои значения.

```
docker compose exec db pg_dump -U app app > backup.sql
```

Восстановление стирает текущую базу и записывает копию на её место. Не запускайте его, если файл копии не открывается. Команды ниже написаны для локального Compose.

```
docker compose stop api
docker compose exec db dropdb -U app --force --if-exists app
docker compose exec db createdb -U app app
docker compose exec -T db psql -U app -d app < backup.sql
docker compose up -d
```

Копию со схемой старше текущего кода API догонят миграции при старте, если в применённые файлы не вносились ручные правки. Если `psql` останавливается на ошибке, не запускайте ту же команду повторно: сохраните текст ошибки и передайте разработчику.

Миграции

Файлы лежат в каталоге `api/migrations`. Применённый файл не правят. Новое изменение схемы записывают новым файлом. Парный файл `down` нужен, чтобы миграция была обратимой по замыслу. На стенде с данными `down` не запускают.

Хранение данных

Данные	Срок по умолчанию	Переменная
Удалённый проект в корзине	30 дней	<code>TRASH_RETENTION_DAYS</code>
Журнал событий симуляции	90 дней (закреплённый не удаляется)	<code>SIM_LOG_RETENTION_DAYS</code>
Сессия без запросов	168 часов	<code>SESSION_IDLE_HOURS</code>
Предельный срок сессии	30 дней	<code>SESSION_MAX_DAYS</code>
Журнал действий администратора	365 дней	нет

Окончательное удаление проекта удаляет и связанные с ним данные.

Если стенд не отвечает

1. Откройте `http://localhost:8080/health`. Ответ 200 значит, что процесс API работает.
2. Посмотрите журнал API:

```
docker compose logs api
```

В строках журнала есть `request_id`, при действиях с проектом также `project_id` и `op`. Пароли и токены в журнал не попадают.

3. Если веб-сервер отвечает, а API нет, интерфейс показывает сообщение «Сервер не отвечает. Повторите через минуту.» (код `api_unavailable`).
4. Если контейнеры остановились после перезапуска Docker, пересоздайте сеть Compose без удаления данных:

```
docker compose down && docker compose up -d
```

Если просите помощи, передайте `request_id` из ошибки, а не снимок экрана с паролями.

Справочник

Переменные окружения

Сервер API читает настройки из переменных окружения. Пример значений для локального запуска лежит в файле `.env.example`. Секреты в репозиторий не кладут.

База данных и сеть

Переменная	По умолчанию	Назначение
<code>DATABASE_URL</code>	нет	Строка подключения к Postgres. Обязательна
<code>API_PORT</code>	<code>8080</code>	Порт API
<code>PGPOOL_MAX_CONNS</code>	<code>10</code>	Размер пула соединений
<code>PUBLIC_ORIGIN</code>	<code>http://localhost</code>	Адрес сайта для проверки заголовка <code>Origin</code> у изменяющих запросов
<code>ENGINE_DIR</code>	<code>/engine</code>	Каталог с <code>engine.wasm</code> и <code>wasm_exec.js</code> , которые API отдаёт браузеру

Сессии и безопасность

Переменная	По умолчанию	Назначение
<code>COOKIE_SECURE</code>	<code>true</code>	<code>true</code> : cookie <code>__Host-session</code> только по HTTPS. <code>false</code> только для локальной работы, тогда cookie называется <code>session</code>
<code>SESSION_IDLE_HOURS</code>	<code>168</code>	Сессия без запросов заканчивается через столько часов (1-8760)
<code>SESSION_MAX_DAYS</code>	<code>30</code>	Предельный срок сессии в днях (1-365)
<code>BCRYPT_COST</code>	<code>10</code>	Стоимость хэширования паролей: 10, 11 или 12
<code>REGISTRATION_MODE</code>	<code>open</code>	<code>open</code> : регистрация открыта. <code>invite</code> : только по приглашению
<code>APP_ENV</code>	<code>local</code>	Значение <code>local</code> открывает <code>GET /api/dev/outbox</code> для просмотра писем. На выкладке задайте другое

Демонстрационные учётные записи

Переменная	Назначение
DEMO_USER_EMAIL	Адрес демо-пользователя, по умолчанию <code>user@demo.local</code>
DEMO_USER_PASSWORD	Пароль демо-пользователя
DEMO_ADMIN_EMAIL	Адрес демо-администратора, по умолчанию <code>admin@demo.local</code>
DEMO_ADMIN_PASSWORD	Пароль демо-администратора

Каталог

Переменная	По умолчанию	Назначение
CATALOG_CSV	<code>/data/catalog.csv</code>	Файл для первой загрузки каталога
CATALOG_SPECS	<code>/data/seeds/robot_specs.json</code>	Файл с ТТХ роботов, применяется при каждом старте

Симуляция и хранение

Переменная	По умолчанию	Назначение
SIM_WORKERS	2	Число обработчиков заданий симуляции в процессе API (1-8)
SIM_JOB_TIMEOUT_MIN	10	Предел времени одного задания в минутах (1-60)
SIM_MAX_ACTIVE_PER_USER	3	Активных заданий на пользователя (1-20)
SIM_LOG_RETENTION_DAYS	90	Срок хранения журнала событий (1-3650)
TRASH_RETENTION_DAYS	30	Сколько дней удалённый проект лежит в корзине (1-3650)

Почта

Переменная	По умолчанию	Назначение
MAIL_MODE	capture	capture : письма остаются в базе и наружу не уходят. smtp : отправка через SMTP_HOST
MAIL_FROM	no-reply@localhost	Адрес отправителя

Переменная	По умолчанию	Назначение
SMTP_HOST	нет	Сервер SMTP. Обязателен при MAIL_MODE=smtp
SMTP_PORT	587	Порт SMTP (STARTTLS)
SMTP_USER , SMTP_PASSWORD	нет	Учётные данные SMTP

Переменные Docker Compose

Переменная	По умолчанию	Назначение
WEB_PORT	80	Порт интерфейса на хосте
API_HOST_PORT	8080	Порт API на хосте
COOKIE_SECURE , PUBLIC_ORIGIN , APP_ENV , MAIL_MODE , MAIL_FROM , REGISTRATION_MODE	см. выше	Передаются в контейнер API. В Compose для COOKIE_SECURE по умолчанию false

Программный интерфейс

Полное описание методов, параметров и схем ответов находится в файле `api/openapi.yaml` (OpenAPI 3.0.3). Этот раздел описывает общие правила и группы методов.

Общие правила

- Обмен идёт в формате JSON. Исключения: выгрузка файлов, изображения роботов и файлы браузерного ядра.
- Адрес API в составе Compose: `http://localhost:8080`. Через веб-сервер те же методы доступны по префиксу `/api`.
- Аутентификация построена на cookie сессии. При `COOKIE_SECURE=true` cookie называется `__Host-session`, иначе `session`.
- Изменяющие запросы должны приходить со страницы приложения: заголовок `Origin` сверяется с `PUBLIC_ORIGIN`. При несовпадении API отвечает ошибкой `csrf`.
- Содержимое проекта меняется только операциями: `POST /api/projects/{id}/transactions`. Методов `PUT` для содержимого проекта нет.
- Расчёт, подбор и симуляция детерминированы при одинаковом снимке проекта, каталоге, версии модели и зерне генератора.

Формат ошибки

Ошибки возвращаются в одной форме:

```
{
  "error": "Текст для человека: что произошло и что делать",
  "code": "stale_result",
  "request_id": "..."}

```

Код	Когда возвращается
<code>csrf</code>	Запрос пришёл не со страницы приложения или страница устарела
<code>forbidden</code>	Не хватает прав на проект или на действие
<code>not_found</code>	Такого адреса или ресурса нет
<code>invalid_value</code>	Значение не прошло проверку схемы
<code>stale_result</code>	Черновик проекта изменился после расчёта, текущий результат выгрузить нельзя (HTTP 409)
<code>api_unavailable</code>	Веб-сервер не получил ответа от API (HTTP 503)

Группы методов

Группа	Основные адреса	Назначение
Служебные	<code>/health</code> , <code>/version</code>	Проверка работы и версии моделей
Каталог	<code>/api/solutions</code> , <code>/api/catalog/dictionaries</code> , <code>/api/catalog/bundle</code>	Список и карточки роботов, справочники, полный набор для демо
Типы объектов	<code>/api/object-types</code> , <code>/api/object-types/{type}/schema</code>	Типы объектов и схемы их параметров
Гость	<code>/api/guest/calculate</code> , <code>/api/guest/match</code> , <code>/api/guest/export</code>	Расчёт, подбор и выгрузка без сохранения
Учётные записи	<code>/api/auth/*</code>	Регистрация, вход, выход, подтверждение почты, сброс пароля, сессии, приглашения
Проекты	<code>/api/projects</code> , <code>/api/projects/{id}</code> , <code>/copy</code> , <code>/restore</code> , <code>/purge</code> , <code>/trash</code> , <code>/import</code>	Создание, копирование, корзина, импорт снимка
Содержимое проекта	<code>/transactions</code> , <code>/snapshot</code> , <code>/operations</code> , <code>/events</code> , <code>/presence</code>	Операции, снимок, журнал операций, поток событий, присутствие
Расчёт	<code>/api/projects/{id}/calculate</code> , <code>/match</code> , <code>/calculations</code> , <code>/api/calculations/{runId}</code>	Расчёт, подбор, сохранённые запуски
Отчёты	<code>/api/projects/{id}/export</code> , <code>/api/calculations/{runId}/export</code>	Выгрузка PDF и XLSX
Участники	<code>/api/projects/{id}/members</code>	Роли в проекте
Карта и симуляция	<code>/map/validate</code> , <code>/map/template</code> , <code>/simulations</code> , <code>/api/simulation-jobs/{jobId}</code> , <code>/api/simulation-runs/{runId}</code>	Проверка карты, задания, результаты, журналы событий
Нормативы	<code>/api/norms</code> , <code>/api/admin/norms</code>	Чтение и изменение нормативов
Администрирование	<code>/api/admin/solutions</code> , <code>/api/admin/catalog/*</code> , <code>/api/admin/invitations</code> , <code>/api/admin/audit</code>	Каталог, загрузка таблиц, приглашения, журнал

Ответ на выгрузку отчёта содержит заголовки `X-Run-Id` и `X-Input-Hash`. По ним отчёт связывается с запуском.

Интеграции

Программные интерфейсы для импорта данных:

- параметры объекта: `POST /api/projects/import` и операции проекта;
- каталог: `/api/admin/catalog/imports` (предпросмотр, подтверждение, список ошибок);
- нормативы: `/api/admin/norms`.

Коннекторов к WMS, ERP, 1C и системам мониторинга в поставке нет. Возможные интеграции описаны в разделе [Ограничения и план](#).

Глоссарий

Термин	Определение
AMR (Autonomous Mobile Robot)	Автономный мобильный робот. Ездит по складу без магнитных лент и рельсов, ориентируясь по датчикам и карте
CAPEX	Капитальные затраты на приобретение и внедрение решения: оборудование, лицензии, монтаж, базовая интеграция
ERP	Система планирования ресурсов предприятия
FMR (Forklift Mobile Robot)	Автономный вилочный погрузчик или штабелёр
IRR	Внутренняя норма доходности: ставка дисконтирования, при которой NPV равна нулю
NPV	Чистая приведённая стоимость: сумма денежных потоков, приведённых к году 0 по ставке дисконтирования
OPEX	Операционные затраты на эксплуатацию: обслуживание, аренда, расходные материалы, энергия, сервисные контракты
RaaS (Robot as a Service)	Аренда роботов с платой за использование, без покупки оборудования
ROI	Отношение накопленного эффекта за период к инвестициям, в процентах
SKU	Единица номенклатуры товара
SLA	Соглашение об уровне обслуживания. В симуляции: допустимая доля заданий, выполненных с нарушением срока
TCO	Совокупная стоимость владения: CAPEX плюс операционные затраты за горизонт с учётом замен
WMS	Система управления складом
БАС	Беспилотная авиационная система
БРС	Беспилотная роботизированная система
Вариант	Состав флота роботов и способ оплаты в проекте
Вердикт	Результат подбора для одной модели: рекомендован, требует проверки или не подходит
Горизонт	Срок расчёта экономики в годах
Демо	Готовый проект, который открывается без входа
Запуск	Неизменяемая запись об одном расчёте или одной симуляции

Термин	Определение
Набор допущений	Именованный набор значений, которые заменяют нормативы в расчёте проекта
Норматив	Значение по умолчанию, которое модель берёт, когда оно не задано в проекте или в карточке робота
Окупаемость простая	Момент, когда накопленный денежный поток достигает нуля. Ставка дисконтирования на неё не влияет
Оценка	Число от 0 до 1, которым подбор упорядочивает модели с одним вердиктом
Сверка	Сравнение производительности из симуляции с производительностью в экономике. Расхождение больше 15% поднимает предупреждение
Сценарий	Один из трёх способов сравнения: база, покупка, RaaS
ТТХ	Технические характеристики робота
Флот	Набор роботов варианта
Чувствительность	Изменение результата при изменении цены оборудования, объёма операций или фонда оплаты труда

Библиотеки и источники данных

Сервер

Библиотека	Версия	Назначение	Адрес
Go	1.27.1	Язык сервера и ядра расчёта	https://go.dev
chi	5.3.2	HTTP-маршрутизатор	https://github.com/go-chi/chi
pgx	5.11.0	Драйвер и пул соединений Postgres	https://github.com/jackc/pgx
sqlc	нет	Генерация кода запросов к Postgres	https://sqlc.dev
golang-migrate	4.20.1	Применение миграций	https://github.com/golang-migrate/migrate
excelize	2.11.0	Чтение и запись файлов XLSX	https://github.com/xuri/excelize
fpdf	0.9.0	Формирование отчётов PDF	https://github.com/go-pdf/fpdf
uuid	1.6.0	Идентификаторы	https://github.com/google/uuid
golang.org/x/crypto	0.57.0	Хэширование паролей (bcrypt)	https://pkg.go.dev/golang.org/x/crypto

Веб-приложение

Библиотека	Версия	Назначение	Адрес
React	19	Интерфейс	https://react.dev
React Router	7	Маршрутизация	https://reactrouter.com
Vite	8	Сборка	https://vite.dev
TypeScript	7	Язык интерфейса	https://www.typescriptlang.org
TanStack Query	5	Запросы к API и кэш	https://tanstack.com/query
Konva и react-konva	10 и 19	Редактор карты и воспроизведение симуляции	https://konvajs.org
pdf.js	6	Чтение файла плана в формате PDF	https://mozilla.github.io/pdf.js

Библиотека	Версия	Назначение	Адрес
SheetJS (сборка @e965/xlsx)	0.20.3	Чтение шаблонов параметров XLSX и CSV в браузере	https://sheetjs.com
IBM Plex Sans, IBM Plex Mono	нет	Шрифты (лицензия OFL)	https://github.com/IBM/plex
Tabler Icons	нет	Значки (лицензия MIT), скопированы в web/src/ui/icons.tsx	https://tabler.io/icons

Инструменты разработки

Инструмент	Назначение	Адрес
Docker и Docker Compose	Запуск платформы	https://docs.docker.com
Postgres 18	База данных	https://www.postgresql.org
nginx	Раздача приложения и проксирование API	https://nginx.org
Playwright	Сквозные тесты	https://playwright.dev
Vitest	Тесты интерфейса	https://vitest.dev
oxlint	Проверка кода TypeScript	https://oxc.rs
mdBook	Сборка документации	https://rust-lang.github.io/mdBook

Данные

Источник	Что взято
Каталог организаторов catalog_export_v4	Названия, производители, цены, отрасли, сценарии
Примеры решений организаторов (DOCX)	Технические характеристики части роботов
Демонстрационные параметры объектов организаторов (XLSX)	Основа демо-проектов и шаблона параметров
Страницы производителей и дистрибьюторов	Дополнительные технические характеристики. Адрес страницы записан в источнике каждого значения в файле data/seeds/robot_specs.json

Правила внесения данных описаны в разделе [Данные и каталог](#).

Ограничения и план

Известные ограничения

Функции

- Симуляция на карте есть только у склада. Для аэропорта и больницы выполняется быстрая проверка без карты.
- Отказы роботов не моделируются.
- Правила диспетчеризации: по порядку поступления, ближайший робот, приоритет SLA.
- Расчёт идёт на собственные средства. Кредит и лизинг не моделируются.
- Цены постоянные, инфляции и роста зарплат нет, остаточная стоимость оборудования не учитывается.
- Ранжирование считается по правилам, а не обучается на данных.

Данные

- Технические характеристики найдены примерно у 55 из 187 роботов каталога. У остальных поля пусты, подбор отмечает их как «требуется проверки».
- Радиус разворота не найден ни у одного робота. Время зарядки у части моделей пустое.
- Срок службы и годовая доля сервиса не найдены ни в одном источнике, расчёт подставляет допущение.
- Посев характеристик не обновляет уже заполненную базу. Новые значения видны на чистой базе или после правки в редакторе администратора.

Эксплуатация

- В составе Compose нет шифрования: HTTPS настраивается на внешнем прокси.
- Нагрузочный тест на 50 одновременных пользователей не проводился.
- Готового развёрнутого стенда в сети нет, платформа запускается командой `docker compose up`.

Интеграции

Интеграций с WMS, ERP, 1C и системами мониторинга в поставке нет. Каталог и параметры загружаются файлами. Обмен с внешними системами идёт через выгрузку PDF и XLSX и через API расчёта.

План развития

Направление	Что планируется
Интеграции	Коннектор к WMS для получения спроса, к ERP и 1C для цен и фонда оплаты труда, к системам мониторинга для фактической доступности роботов

Направление	Что планируется
Каталог	Разбор сайтов поставщиков, плановое обновление каталога, расширение набора ТТХ
Модели	Симуляция на карте для аэропорта и больницы, отказы и обслуживание роботов, встречное движение на двухполосных участках
Экономика	Заёмное финансирование, налог на прибыль, инфляция
Ранжирование	Оптимизация и машинное обучение, когда накопятся данные внедрений
Визуализация	Трёхмерное представление объекта
Роли	Роль вендора

Для разработчиков

Разработка

Раздел для тех, кто меняет код. Правила оформления кода, комментариев, текста интерфейса и документации собраны в файле `AGENTS.md` в корне репозитория. Ниже описано, где что лежит и что нельзя разъединять.

Где что лежит

Числа считают пакеты `api/internal/matching`, `api/internal/econ` и `api/internal/sim`. Интерфейс формулы не придумывает. Сборка прогона симуляции из черновика находится в `api/internal/simbuild`: она не обращается к базе, и один и тот же вход уходит в очередь заданий и в браузерное ядро.

Что	Где
Процесс API	<code>api/cmd/api</code>
Браузерное ядро (WebAssembly)	<code>api/cmd/engine-wasm</code> , файлы отдаются с <code>/api/engine/</code>
Запись проекта операциями	<code>api/internal/ops</code> (эталон), <code>web/src/store</code> (то же применение на клиенте)
HTTP-клиент интерфейса	<code>web/src/api</code>
Запросы к Postgres	<code>api/internal/db</code> , код генерируется sqlc и руками не правится
Загрузка каталога	<code>api/internal/importers</code>
Отчёты	<code>api/internal/exporters</code>
Очередь симуляций	<code>api/internal/jobs</code> , <code>api/internal/simjobs</code>
Схемы параметров объектов	<code>api/internal/objects</code>
Контракты данных	<code>contracts/</code>

Запись проекта идёт только операциями: вставка, установка значения, удаление и перемещение. Отдельного метода `PUT` для содержимого проекта нет. Расчёт операцией не является, его считает Go.

Что нельзя разъединять

- Расчёт при том же снимке, каталоге, версии модели и зерне генератора даёт те же числа. На запуске записывается контрольная сумма каталога. Смена версии экономики или подбора делает сохранённый запуск устаревшим.

- Если симуляция и экономика расходятся больше чем на 15%, в ответе есть флаг проверки. Скрывать его нельзя.
- Коммерческие данные гостя в Postgres не пишутся.
- Загрузка строки каталога идемпотентна по идентификатору решения.
- Версия экономики задана константой `econ.ModelVersion` (сейчас `econ-v3`). В ADR 0004 она ещё названа `econ-v2`, действует константа. Версия событийной симуляции: `sim.Version` (`sim-v2`). Быстрая проверка пропускной способности внутри расчёта остаётся `sim-v1` в `api/internal/sim/geom`.

Контракт HTTP: `api/openapi.yaml`. Почему нет `PUT` и как устроены снимки: [ADR 0001](#) и [ADR 0003](#).

Новое поле проекта

Поле начинается в `contracts/ops/project.schema.json`. Затем из корня репозитория выполните:

```
node scripts/gen-ops-schema.mjs
```

Скрипт копирует схему к читателям в Go и в `web/src/store` и обновляет сгенерированный Type-Script. Фикстура для поля нужна в `api/internal/ops/fixtures_test.go`. Пока фикстуры нет, поле считается незакрытым.

Новый маршрут API

Обработчик и правка `api/openapi.yaml` идут в одном изменении. Поля ответа в Markdown не дублируют, их описывает `api/openapi.yaml`.

Новая переменная окружения

Чтение идёт в `api/internal/config`. Имя и смысл записываются в `.env.example` и в раздел [Переменные окружения](#). Секреты в репозиторий не кладут.

Новая миграция

Новое изменение схемы записывают новым файлом в `api/migrations`. Применённый файл не правят. Если добавляется индекс, причина указывается в имени файла.

Тесты

Тесты сервера:

```
cd api && go test ./...
```

Интеграционные тесты (очередь симуляций, права, гость, отчёты, миграции вверх и вниз) запускаются, если задана переменная `TEST_DATABASE_URL`. Пользователю нужно право `CREATE DATABASE`, каждый тест создаёт и удаляет свою базу:

```
docker run -d --name sim-it-pg -e POSTGRES_USER=app -e POSTGRES_PASSWORD=app -p 55432:5432
postgres:18
cd api && TEST_DATABASE_URL="postgres://app:app@localhost:55432/app?sslmode=disable" go test
./...
```

Тесты интерфейса:

```
cd web && npm test && npm run lint && npm run build
```

Сквозные тесты Playwright выполняются на чистом стеке Compose. Отдельный проект Compose не трогает основной том. `PUBLIC_ORIGIN` должен совпадать с `E2E_BASE_URL`, иначе API отклонит запросы с кодом `csrf`:

```
PUBLIC_ORIGIN=http://localhost:8088 WEB_PORT=8088 API_HOST_PORT=18080 docker compose -p
mh26e2e up --build -d
cd web && npx playwright install chromium && E2E_BASE_URL=http://localhost:8088 npm run e2e
WEB_PORT=8088 API_HOST_PORT=18080 docker compose -p mh26e2e down -v
```

Документация

Документация лежит в каталоге `docs/` и собирается mdBook. Каталоги `docs/book/` и `docs/export/` собираются заново, `docs/book/` в git не коммитят.

Сайт для просмотра:

```
cd docs && mdbook serve
```

mdBook ставится командой `cargo install mdbook` или пакетом системы. В образ API и в Compose он не входит.

Единый документ в форматах PDF и DOCX:

```
node scripts/docs/build.mjs
```

Сценарий собирает главы в порядке оглавления и создаёт файлы `docs/export/documentation.pdf` и `docs/export/documentation.docx`. Нужны Pandoc и Google Chrome.

Текст документации следует правилам `AGENTS.md`: без тире и стрелок, с буквой «ё», кавычки «ёлочки», десятичная запятая. Внутренние коды (версии моделей, идентификаторы полей, контрольные суммы) в пользовательских разделах не показываются.

ADR 0001. Снимки, варианты, задания и события

Статус: принято Дата: 2026-09-16

Контекст

Текущая модель хранит изменяемые `projects.params` и перезаписывает `projects.results`. Изменение входа не помечает результат устаревшим. Один робот и три финансовых kind живут в одном JSON. Симуляция синхронна и не сохраняет журнал.

Кандидат к 2026-09-29 показывает один складской сценарий. Полный пилот после сдачи.

Решение

Снимки

- Черновик проекта изменяемый. Канонический документ: `ProjectDraft`.
- Расчет создает неизменяемые `project_versions` (`ProjectSnapshot`) и `calculation_runs`.
- Сервер считает `input_hash` = SHA-256 канонического JSON. В hash входят params, процессы, варианты, флот, финансирование, карта, ревизия каталога, выбранные id, econ overrides, версии match/econ/sim.
- Порядок ключей и неупорядоченных массивов нормализуется сервером. `sort_order` процессов и вариантов сохраняется. Params хешируются так, как их читает схема (`objects.NormalizeParams`): миграция, которая меняет только формат хранения, hash не меняет.
- Seed запуска в hash не входит. Seed хранится на запуске.
- Если hash черновика не равен hash текущего запуска: результат `stale`, экспорт текущего результата блокируется, исторический запуск открывается явно.
- С запуском сравнивается `calculation_runs.draft_hash`: снимок запуска, посчитанный текущим определением hash. `input_hash` запуска остается тем, что записан при расчете, и идет в отчеты. Смена определения hash сама по себе не делает запуск устаревшим: миграция обнуляет `draft_hash`, API пересчитывает его из снимка при следующем сохранении черновика. Запуск другой версии econ или match остается устаревшим. Тест `TestInputHashGolden` падает при любой смене hash.
- `projects.results` остается кэшем последнего запуска. Источник истины: `calculation_runs` + `calculation_results`.

Варианты

- Процессы принадлежат проекту и образуют общую базу.
- `solution_variants` задают технический состав. `variant_fleet_items` допускают смешанный флот.
- `financing_scenarios` отделены от флота. Один вариант сравнивает buy и тарифы RaaS без копирования состава.
- Одновременно до шести вариантов, по умолчанию три черновика.

- Не использовать таблицу `scenarios` из 0001. Она не часть этой модели.

Jobs

- Длинная симуляция идет через `simulation_jobs` и встроенный worker pool в том же API-процессе.
- HTTP не ждет прогон. Клиент читает статус по `job_id`.
- Отмена, повтор и квота по пользователю или проекту. Без Redis и отдельных воркеров.
- Реализация очереди: 22-24 сентября. Контракт фиксируется сейчас.

Событийное ядро

- `sim-v1` остается геометрическим прогоном до замены.
- `sim-v2` (22-24 сентября) пишет компактный журнал `SimulationEvent`.
- Сервер отдает события и контрольные состояния, не покадровый снимок сцены.
- Типы событий склада: `job_arrival`, `dispatch`, `route_start`, `route_end`, `resource_acquire`, `resource_release`, `load`, `unload`, `charge_start`, `charge_end`, `sla_reached`, `sla_violated`.
- Профили AMR и паллетного класса не сводятся к одним коэффициентам.

Карта

- Черновик: `MapDocument` в JSONB. Исходный PNG/JPEG/PDF на сервер не пишется.
- Снимок карты входит в `ProjectSnapshot` и неизменяем.
- Профиль склада: `indoor`.

Последствия

- Backend и frontend реализуют один контракт из `api/openapi.yaml` и `contracts/jsonschema/`.
- Миграции только forward-only.
- Нельзя убирать hash, stale, неизменяемый запуск и серверную валидацию.

ADR 0002. Симуляция, карта и очередь

Статус: принято Дата: 2026-09-16

Уточняет ADR 0001 для этапов 22-26 сентября.

Версии

- `sim-v1` переехал в `internal/sim/geom`. Он остается быстрой проверкой расхождения экономики и симуляции внутри `/calculate`. Гостевого `/api/guest/sim` больше нет: демо запускает `sim-v2` в браузере. Результаты `sim-v1` помечены `sim-v1`, а не `econ-v1`.
- `sim-v2` живет в `internal/sim`: событийное ядро, профили роботов в `internal/sim/profiles`.
- `calculation_runs.sim_version` расчета экономики остается `sim-v1`. Запуск симуляции пишет `sim-v2` и `kind = simulation`.
- Результат симуляции устаревший, если `hash` черновика отличается от `hash` запуска или версия `sim-v2` сменилась.

Модель

- Задание перевозит груз от точки забора к точке доставки процесса. Точки задаются потоками `MapDocument.layers.flows`, а не `project_processes.point_ids`.
- Спрос в час = спрос в сутки / единиц в задании / активные часы суток x профиль часа. Активные часы берутся из окна смены.
- Профили AMR и паллетного робота различаются действиями (подъем под грузом или вилы с подъемом на ярус), габаритами, скоростью с грузом и потреблением энергии. Значения каталога переопределяют значения класса, подставленные значения перечисляются в допущениях.
- AMR с грузоподъемностью от 400 кг возит паллеты по полу, но не ставит их в стеллаж. Размещение и отгрузка из ячеек выполняет паллетный робот.
- Общие ресурсы: док (точки), узкий участок (ребра), зарядка (точки). Очередь к ресурсу FIFO.
- Однополосный участок: ширина меньше $2 \times \text{ширина} + 1.5 \times \text{зазор}$ самого широкого класса. Цепочки однополосных ребер без развилок становятся ресурсом вместимостью 1. Робот, стоящий в тупике участка, держит его.
- Ширина ребра = заявленная ширина, ограниченная свободным коридором до препятствий и стен рабочей области. Ребро уже $\text{ширина} + \text{зазор}$ класса непроходимо для него, маршрут идет в объезд.
- Диспетчеризация: `fifo`, `nearest`, `sla_priority`. SLA: ожидание до назначения и полный цикл.
- Вероятностный режим: пуассоновский поток, логнормальные длительности, до 30 повторов с `seed seed + i`. Детерминированный режим: один прогон.
- Итог: медиана, `min`, `max`, `p10`, `p90`. SLA выполнен, если `p90` доли нарушений не выше допуска (5% по умолчанию).
- Узкое место: сначала насыщенный ресурс (загрузка от 85% или средняя очередь от половины вместимости), затем флот, если роботы свободны меньше 10% времени.

Журнал

- Хранится один повтор, ближайший к медиане нарушений и производительности. Он перезапускается с записью, метрики совпадают с прогоном без записи.
- Формат `sim-log-v1` : заголовок со сценой в метрах, графом, роботами и ресурсами; события `sim-event-v1` ; контрольные состояния каждые 60 с.
- Журнал сжат gzip и лежит в `simulation_artifacts` . Срок хранения 90 дней, закрепленный журнал не удаляется. Лимит 400 000 событий, дальше `truncated` .

Очередь заданий

- Таблица `simulation_jobs` , захват `FOR UPDATE SKIP LOCKED` , встроенный пул в процессе API (`SIM_WORKERS`).
- Heartbeat раз в секунду пишет прогресс и проверяет отмену. Зависшее задание без heartbeat 30 с возвращается в очередь или падает после `max_attempts` .
- При остановке API текущее задание возвращается в очередь без списания попытки.
- Итог, повторы, журнал и статус `succeeded` пишутся одной транзакцией.
- Одно активное задание на проект, `SIM_MAX_ACTIVE_PER_USER` на пользователя.

Карта

- Координаты объектов в пикселях страницы, метры = пиксели x `meters_per_px` . Исходный план не отправляется на сервер.
- Черновик карты сохраняется даже с ошибками, ошибки пишутся в документ. Запуск симуляции с ошибками карты запрещен.
- Без карты проекта используется шаблон склада 84 x 52 м, уровень подтвержденности `preliminary` и предупреждение. С картой проекта уровень `configured` .

Симуляция в браузере

- Сборка запуска (`JobConfig` , `Build` , шаблон и проверка карты, `Summary`) живёт в `internal/simbuild` , без базы и HTTP (это проверяет `deps_test.go`). Очередь заданий (`internal/simjobs`) и браузерное ядро (`api/cmd/engine-wasm`) строят запуск одним кодом. Флот берётся из нейтральной карты `Robots` , её даёт `simbuild.RobotsOf(engine.Catalog)` .
- Ядро в браузере отвечает `simulateMap` , `checkMap` , `mapTemplate` и `draftHash` . Запуск получает каталог и записи проекта (`collections`), возвращает сводку и журнал одного повтора. Сводка та же, что у серверного запуска, кроме `finished_at` и `duration_ms` ; совпадение с нативной сборкой держит `contracts/engine/reference.json` .
- Запуски демо лежат в IndexedDB `robots-demo-runs` : пять последних на демо, журнал отдельно от сводки. Запуск устарел, если хеш записей демо теперь другой (`draftHash`). Запуск идёт по одному; отмена останавливает рабочий поток симуляции, расчёт живёт в другом потоке.
- Демо-склад собран заранее: `web/src/guest/demo-warehouse.json` , его пишет `go test ./internal/simbuild -run TestDemoWarehouse -update` . Карта 84 x 52 м (шаблон `simbuild.Template`) и три варианта с роботами из каталога. Спрос равен минимуму схемы: на

четырёх доках карты проходит около 500 паллет в сутки, при 1000 любой вариант упирается в них. Число роботов подобрано симуляцией. Тест требует вердикт `pass` у каждого варианта и расчёт без флага сверки; если каталог или значения по умолчанию изменились, он сообщает, что файл устарел.

Не сделано

- Автоматическое построение графа, встречное движение внутри двухполосного ребра, лифты и двери.
- Отказы и обслуживание: нет данных, модель работает без отказов.
- Перенос производительности из симуляции в экономику через `apply-simulation-throughput` снят. Размер флота подбирается прогонами `sim-v2` (`JobConfig.Mode = fleet_search`) и записывается в вариант кнопкой «Взять N».

ADR 0003. Операции и синхронизация

Статус: принято Дата: 2026-09-19

Основание: совместная работа над проектом без сети и с несколькими редакторами.

Решения

Вопрос	Решение
Совместная работа	несколько редакторов одновременно и долгая работа без сети
Роли	<code>owner</code> , <code>editor</code> , <code>viewer</code> на проект
Модель данных	таблицы сущностей остаются; изменения идут операциями
Операции	<code>insert</code> , <code>set</code> , <code>delete</code> , <code>move</code> над коллекциями из <code>contracts/ops/project.schema.json</code>
Интерпретатор	<code>api/internal/ops</code> (эталон) и <code>web/src/store/apply.ts</code> ; фикстуры создает Go, TS получает тот же результат
Клиент	кэш в памяти и IndexedDB, очередь транзакций, оптимистичное применение
Уведомления	SSE от сервера, POST от клиента, LISTEN/NOTIFY между экземплярами API
История	undo и redo только в сеансе, обратными транзакциями
Конфликт после офлайна	транзакция не применяется и попадает в список "Не применено"
Расчеты	только Go на сервере, это не операции
Главный источник	сервер

Не вводим: CRDT, WebSocket, Redis, Kafka, `apply()` через WASM, отдельный код каждой операции на двух языках.

Словарь схемы

Словарь фиксирован. Новое слово только через новый ADR. Требование, которое словарем не выражается, становится проверкой на Go перед расчетом или макросом в TS.

- Коллекция: `kind` (`singleton` или `list`), `ordered` , `max_items` , `unique` , `fields` , `requires` .
- Типы полей: `string` , `int` , `number` , `bool` , `enum` , `point` , `points` , `object` , `params` , `ref` , `ref_list` , `external_ref` .

- Ограничения: `min`, `max` (числа), `min_length`, `max_length` (символы строки, элементы `points` и `ref_list`), `nullable`, `required`, `immutable`, `values` (`enum`), `validator` (имя проверки для `object`).
- Ссылки: `to`, `key` (по умолчанию `id`; иначе неизменяемое уникальное строковое поле цели), `on_delete`: `cascade` и `set_null` для `ref`, `remove` для `ref_list`.
- `requires`: `{"if_set": "a", "then_nonempty": "b"}`. Задано: не `null`. Непусто: не `null`, строка не из одних пробелов, непустой список.

Смысл примитивов

- Запись: `id`, `order` у упорядоченной коллекции, затем все поля схемы.
- Singleton: одна запись с `id`, равным имени коллекции. Существует всегда, `insert` и `delete` отклоняются с `immutable`. В `set id` можно не указывать.
- `insert`: `id` от клиента, UUID в нижнем регистре. Коллекции карты (`legacy_ids`) принимают и `id` вида `p1` (1-64 символа: латиница, цифры, `-`, `_`, `.`): такие `id` были у карт до операций, и отмена удаления возвращает запись под прежним `id`. Новые записи клиент создает только с UUID. Не указанное поле получает `null`, если поле `nullable`, иначе нулевое значение типа (`""`, `0`, `false`, `[]`, `{}` для `params`, `{x: 0, y: 0}` для `point`). Поле с `required` указать обязательно. `id` в значении и неизвестные поля отклоняются.
- `set`: новое значение поля или поле.ключ для `object` и `params`. Путь глубже одного ключа отклоняется. Равное текущему значению ничего не меняет, транзакция применяется.
- `delete`: удаляет запись и выполняет `on_delete` у всех ссылок на нее, рекурсивно.
- `move`: новый ключ порядка упорядоченной записи.
- Ключ порядка: дробная индексация, алгоритм `npm fractional-indexing` (`api/internal/ops/order.go`, `web/src/store/order.ts`). Порядок записей: ключ, затем `id`. `sort_order` в таблицах и черновике равен позиции записи.

Проверки

Порядок проверок одинаков на Go и TS, фикстуры его закрепляют.

1. На каждой операции: вид операции и коллекция, существование цели (`target_missing`), путь, тип и ограничения значения, `immutable`, именованный валидатор `object`.
2. После всех операций, по записям в порядке первого изменения: ссылки (`ref_missing`), `external_ref` через хук, `params` через хук, `requires`. Проверяются только поля, заданные операциями: каскад не проверяет старые ссылки записи.
3. Затем по коллекциям, где были `insert` или `set`: `max_items` (`limit`, только при вставке), `unique` (только для записей, у которых менялось поле набора). `null` в `unique` равен `null`.

Любое нарушение отклоняет всю транзакцию. Результат отказа: причина и `{op, coll, id, field}`.

Изменения (Changes)

Применение возвращает чистый итог по каждой затронутой записи в порядке первого касания:

`insert` с `after`, `delete` с `before`, `update` с `before` и `after`. Каскад входит в итог. Из него строится обратная транзакция `undo`.

Хуки

- `params`: поле типа `params` проверяется схемой типа объекта записи (`object_type`, `api/internal/objects`). Проверяются заданные ключи, при замене всего поля все ключи. Смена `object_type` проверяет все поля `params` записи целиком: параметры прежнего типа не проходят, их заменяют в той же транзакции.
- `object`: именованные валидаторы (`process_demand`, `process_sla`, `process_durations`, `process_staff`, `financing_assumptions`, `econ_overrides`, `solution_ids`, `map_page`, `map_segment`).
- Объекты, которые таблицы хранят через структуру Go (`process_demand`, `process_sla`, `process_durations`, `process_staff`, `map_page`, `map_segment`), сервер принимает только в хранимой форме: ключ с нулевым значением у поля с `omitempty` отсутствует, остальные ключи есть всегда. Иначе следующий снимок отличался бы от состояния клиента. Обнулить такой ключ можно только заменой всего объекта.
- `external_ref`: решение каталога в закреплённой ревизии проекта.
- Клиент, который не может проверить значение, пропускает его, решает сервер.
- Фикстуры используют заглушки из `contracts/ops/fixtures/hooks.json`: `params` по таблице типа объекта; `object` проверяет разрешённые ключи (`number` не меньше 0, `string`, `any`) или список строк; `external_ref` по списку id.

Проект в коллекциях

- `project`: имя, тип объекта, параметры, выбор подбора, переопределения экономики, активный набор допущений (`active_assumption_set_id` вместо флага `is_active` у набора).
- `processes`, `variants`, `fleet_items`, `financing`, `shared_costs`, `assumption_sets`: строки таблиц с теми же id.
- `map` (singleton): страница, профиль, масштаб, отрезки калибровки и проверки. Карта есть, если `map.page` не `null`.
- `map_points` (координата `pos` одним значением, чтобы два перемещения не смешали `x` и `y`), `map_edges`, `map_zones`, `map_obstacles` (упорядочены: порядок отрисовки), `map_resources`, `map_flows`. Поток получил постоянный `id` в документе карты; у старых карт он выводится из кода процесса.
- `ops.FromDraft` и `ops.ToDraft` переводят черновик в коллекции и обратно. Сообщения проверки карты (`errors`, `warnings`) пишет сервер при сохранении.

Сервер

Решения реализации:

- `POST /api/projects/{id}/transactions` работает под блокировкой строки проекта: пакки в один проект идут по очереди, `seq` растёт без пропусков. Состояние собирается из таблиц (`ops.FromDraft`), после пакки таблицы затронутых коллекций переписываются целиком с `id` и ключами порядка записей, затем черновик и `input_hash` пересчитываются. Сообщения проверки карты пересчитываются, только если менялась карта. Смена `object_type` отцепляет текущий запуск и результат: запуск остается в истории, но отвечает на вопрос о другом объекте.
- Повтор `tx_id` возвращает сохраненный исход. Неразборчивая операция отклоняет свою транзакцию с `invalid_value`, а не всю пакку: иначе одна плохая запись остановила бы очередь клиента.
- Числа в таблицах без масштаба (`NUMERIC`), чтобы значение из операции сохранялось точно.
- Журнал `project_operations` хранит и отклоненные транзакции (без `seq`, с причиной). Записи сервера имеют `client_id = "server"`: расчет и подбор меняют выбор решений и переопределения через `set` с меткой `calculate` или `match`, только если значение изменилось. Расчет пишет `calculation_runs.draft_seq`; если черновик изменился за время расчета, черновик не трогается и новый запуск сразу устаревший.
- PUT содержимого проекта больше нет: все правки идут транзакциями. `reset` в журнале остался один: копия проекта (`logReset(ctx, id, "copy")`), которая заполняет новый проект мимо операций; клиенты, дойдя до такой записи, берут снимок.
- Проект получает процессы, варианты и наборы допущений по умолчанию, только пока `draft_seq = 0`. После первой правки пустой список это выбор пользователей.
- `POST /api/projects/import` создает проект из снимка браузера: `ops.Rekey` выдает записям новые идентификаторы (старые принадлежат тому проекту, откуда снимок), `ops.Replace` превращает разницу между заготовкой нового проекта и снимком в одну транзакцию, и она проходит те же проверки схемы, что обычная правка. Создание и заполнение идут в одной транзакции базы: снимок, который схема не приняла, не оставляет ни проекта, ни записей.
- `GET /snapshot` отдает `{schema_version, seq, collections}`, `GET /operations?after=` принятые транзакции по порядку или `reset: true` (в диапазоне `reset`, часть журнала старше 30 дней или `after` больше `seq`).
- SSE `GET /events`: пакет `api/internal/realtime`, одно соединение `LISTEN` на процесс вне пула. `runs`, `access` и `deleted` шлют триггеры Postgres (миграция 0014), поэтому их видят все экземпляры API и все пути записи, включая воркеры. После потери соединения `LISTEN` каждый поток получает `reset`. Маршрут вне таймаута запроса, сессия проверяется на каждом heartbeat.
- Присутствие: `POST /presence` рассылается через `NOTIFY`, каждый процесс держит список в памяти. Запись ключуется `user_id/client_id`, так что чужой `client_id` не затрагивает запись другого человека, и живет 90 с (фоновая вкладка получает таймер раз в минуту).

Клиент

- `web/src/store/`: хранилище проекта (`confirmed` и очередь `pending`, вид со структурным разделением), синхронизация с паузой от 1 до 60 с, IndexedDB `robots-store`, один поток SSE на браузер через `Web Lock` и `BroadcastChannel`. Идентификатор вкладки (`clientId`, ключ очереди) лежит в `sessionStorage` только пока страница ушла (`pagehide`, `freeze`): перезагрузка находит его и сохраняет очередь, а «Дублировать вкладку» копирует хранилище живой страницы,

где его нет, и получает свой. Список «Не применено» хранится по тому же ключу проект, человек, вкладка; очередь и список закрывшейся вкладки забирает соседняя вкладка того же человека. Хранилище и поток живут, пока открыта любая страница проекта: их держит статус сохранения в верхней панели.

- Карта (`web/src/map/records.ts`): документ строится из записей так же, как `toMap` на сервере (совпадение проверяет `contracts/ops/map_document.json`). Команда редактора превращается в операции одной транзакции; удаление точки дает одну операцию, остальное удаляет каскад схемы. Перетаскивание рисуется локально, транзакция уходит при отпускании мыши. Координаты округляются до 0,01 пикселя, как при старом сохранении. Шаблон и загруженный документ заменяют карту одной транзакцией и сохраняют свои `id`.
- Новые объекты карты получают `UUID` и название-код (`T4` , `e2` , `z3` , `res-1`), пронумерованное как `id` до операций. Объекты старых карт показывают свой `id`. Сообщения проверки карты называют объекты так же; ребро с `UUID` называется по концам.
- Undo и redo (`web/src/store/undo.ts`): стек на вкладку и проект, только свои транзакции. Шаг хранит изменения своей транзакции, отмена отправляет обратную транзакцию против текущего состояния, поэтому чужие правки других полей остаются. Запись, которую шаг вставил, а другой уже удалил, не трогается; изменение записи, которую другой удалил, не применяется и попадает в "Не применено". Сдвиги стрелками одной точки подряд образуют один шаг. Так же, как в Figma, отмена возвращает значение своей вкладки поверх более новой чужой правки того же поля, а повтор возвращает чужое: обратная транзакция всегда пишет значение, которое было в момент шага.
- Варианты (`web/src/fleet/records.ts`): правка одного поля дает одну транзакцию с понятной меткой ("Добавить робота", "Убрать робота"). Порядок флота хранится ключами порядка: запись, оставшаяся на своем месте, ключ не меняет, вернувшаяся на прежнее место получает ключ между соседями. Копия варианта вставляет вариант, флот и финансирование с новыми `id`. Удаление варианта это одна операция, флот и финансирование удаляет каскад.
- Поле формы сохраняется через 600 мс после ввода, по Enter и при выходе из поля (`D1`), общий хук `web/src/ui/fieldDraft.ts` . Значение, которое схема не примет (пустое имя), остается у поля и не отправляется. Диапазоны параметров объекта схема операций оставляет серверу, поэтому форма проверяет их сама (`web/src/params/check.ts`) перед записью: иначе значение вне диапазона применилось бы на месте и вернулось отказом позже.
- Гость (`web/src/store/guestSync.ts`): проект `guest` живет в том же хранилище, но за ним нет сервера, поэтому каждая принятая транзакция подтверждается на месте и состояние сразу пишется в `IndexedDB`. Строка в верхней панели ("Сохранено в браузере") следует за записью, а не за правкой: закрыть вкладку безопасно ровно тогда, когда она это говорит. Первый снимок собирается из умолчаний схемы типа и ключей `guest-*` прежних версий, после чего ключи удаляются. Другие вкладки того же демо получают каждую транзакцию через `Broadcast-Channel` `robots-demo-<тип>` и применяют её как свою следующую; то, что уже не применяется (запись удалена здесь), пропускается.
- Расчёт демо: `POST /api/guest/calculate` и `/api/guest/export` принимают `schema_version` и `collections` , проверяют их как импорт проекта и считают по черновику из них (процессы, варианты с флотом, финансирование, допущения, общие расходы, что-если). Ничего не сохраняется. Браузерное `calculate` принимает те же `collections` . Ключ результата демо включает отпечаток записей (`web/src/store/fingerprint.ts`), поэтому правка процесса, флота или

допущения пересчитывается. Демо склада сеется из `demo-warehouse.json`; его `version` (хеш записей) хранится вместе со снимком в IndexedDB, и демо со старой версией сеется заново: сервера у демо нет, мигрировать нечего.

- Браузерное ядро (`web/src/engine/`): страница просит сервер и переходит на движок в браузере, только когда запрос до сервера не дошел; такой результат помечен "Предварительно", не сохраняется и не дает выгрузить отчет. Сборка `VITE_APP_MODE=demo` сервера не имеет вовсе и считает всегда в браузере, поэтому отметку не показывает.
- Отказ сервера (`web/src/store/rejected.ts`, `web/src/layout/RejectedList.tsx`): транзакция, которую сервер не принял, остается записью с названием действия, причиной по-русски и временем. "Восстановить" пишет ту же правку заново против текущего состояния: запись, которой уже нет, вставляется с прежними значениями, существующая получает поля, которые правка меняла. В момент отказа выходит toast. Если запись удалил другой участник, строка называет его почту (`actor_email` в `/operations`), а свои удаления из другой вкладки помечает «удалено в другой вкладке».
- Присутствие (`web/src/store/presence.ts`): страница сообщает о себе на `POST /api/projects/{id}/presence` при смене места и раз в 15 с, при возврате из кэша или из фона сообщает снова, а при закрытии прощается с `keepalive`. Идентификатор присутствия новый на каждую загрузку (`pageId()`, не общий с очередью правок), поэтому прощание старой страницы не стирает запись перезагруженной. Место (`Layout`) и выделение (`useReportSelection`, при уходе со страницы сбрасывается) сообщаются отдельно, на любой странице проекта. Сервер шлет список событием `presence ( {"list": [...]})`. Свои же вкладки (тот же `user_id`) в лица не попадают, вкладки другого человека сводятся в одно лицо. В демо присутствия нет. Цвет закреплен за человеком хешем его id, четыре токена `--presence-1..4` на тему.

ADR 0004. Экономика по денежному потоку

Статус: принято Дата: 2026-09-29

Примечание: решение принято для версии `econ-v2`. Действующая версия модели `econ-v3` (`econ.ModelVersion`), суть решений сохранена.

Основание: разбор расчёта двумя независимыми проверками и ответы организаторов (`otvety_ot_orgov.md`, блоки 1, 2, 4 и 5). Организаторы оценивают разумность и полноту допущений и прозрачность расчёта, поэтому каждая цифра должна выводиться из расписания по годам.

Решения

Вопрос	Решение
Основная метрика	простая окупаемость, как просили организаторы; CAPEX считается один раз
Расписание	годовые периоды в постоянных ценах базового года, вложения в году 0, потоки в конце года; инфляции нет
Окупаемость за горизонтом	расписание идёт 30 лет, окупаемость дольше горизонта не экстраполируется, а находится в нём
Дополнительные метрики	NPV, IRR, дисконтированная окупаемость, классический ROI за горизонт, затраты за горизонт (TCO)
Ставка дисконтирования	поле набора допущений <code>discount_rate</code> , по умолчанию 15%; простую окупаемость не меняет
Замена	робот покупается заново, когда срок его службы заканчивается внутри горизонта (только покупка)
Остаточная стоимость	не учитывается: ни робот, доживший до конца горизонта, ни повторная покупка не дают возврата. NPV, ROI и затраты за горизонт получаются консервативными, особенно при повторной покупке незадолго до конца горизонта
Батарея	15% цены робота в годах 4, 8 и далее, пока они внутри срока службы и горизонта; в OPEX среднее не размазывается, в расписании это отдельная сумма в своём году
OPEX	<code>opex_year_rub</code> только повторяющиеся расходы; батареи и замены показаны отдельными строками и в таблице по годам
RaaS	оборудования, батарей и замен нет; плата идёт весь горизонт; в формулах фактические доля и тариф за операцию

Вопрос	Решение
Флот	срок службы, сервис, батарея и цена считаются по каждой позиции флота; результат не зависит от порядка роботов
НДС	ставка по умолчанию 22% (с 2026-01-01); одна политика для всех строк с НДС (оборудование, инфраструктура, лицензии, энергия, расходники, общие затраты), зарплата НДС не несёт
Амортизация	база: роботы по своим срокам, инфраструктура, ПО, интеграция, пусконаладка; обучение расходы, резерв не актив; у RaaS роботов в базе нет
ФОТ	коэффициент взносов остаётся параметром объекта; предельная база 2 979 000 ₽ на человека в год, выше неё 15,1%
Один расчёт затрат	<code>costModel</code> в <code>econ/costs.go</code> используется и <code>evaluate</code> , и <code>evalVariant</code> , поэтому Роботы и Итог не расходятся

Не моделируем

Налог на прибыль, инфляцию и рост зарплат, разгон внедрения (экономия с года 1), финансирование. Это написано в допущениях расчёта.

Последствия

- `ModelVersion = "econ-v2"`. Все сохранённые расчёты считаются устаревшими и пересчитываются; хэш входа проекта меняется (в него входит набор допущений).
- Миграция 0029: колонка `discount_rate` (15%), ставка НДС по умолчанию 22%, наборы со ставкой ровно 20% переведены на 22%.
- В результате сценария убраны `cash_payback_years` и `accounting_roi_pct`: они дублировали `payback_years` и `roi_pct`. Добавлены `discounted_payback_years`, `npv_rub`, `irr_pct`, `cash_flow`.
- Формулы на странице строятся из тех же параметров, что и расчёт (`econ.Formula`); текст и MathML меняются вместе. PDF и XLSX формул и показателей дисконтирования не содержат: это короткие отчёты для решения, простая окупаемость в них главная метрика.
- Числа фиксируются в таблицах тестов в точных рублях, посчитанных независимо от реализации.

Открытые допущения

Ставка 15%, предельная база взносов 2026 года и подразумеваемый тариф за операцию у RaaS (плата, делённая на годовые операции при плановом объёме) выбраны исполнителем. Заказчик может заменить любое из них, не меняя структуру расписания.